SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Scalable Fault-Tolerant Quantum Compiler for Chiplet Architectures

Peter Wegmann

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Scalable Fault-Tolerant Quantum Compiler for Chiplet Architectures

Skalierbarer, fehlerkorrigierender Quantencompiler für Chiplet-Architekturen

Author:	Peter Wegmann
Supervisor:	Prof. Pramod Bhatotia
Advisor:	Aleksandra Świerkowska, Emmanouil Giortamis
Submission Date:	09.02.2026

I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 09.02.2026 Peter Wegmann

Abstract

Quantum Computing has the potential to solve complex problems in the near future, but current quantum hardware faces the severe challenge of overcoming inherent physical noise, which limits both the depth and the width of executable circuits. To achieve error rates low enough to run algorithms with proven quantum advantage, specialized quantum error correction (QEC) codes tailored to quantum systems are necessary. Given the substantial qubit overhead introduced by QEC codes - often several orders of magnitude greater than uncorrected systems - recent superconducting hardware designs have shifted toward modular chiplet architectures to overcome scalability limits of monolithic architectures. To execute quantum algorithms, compilers must account for the increased hardware limitations to ensure all physical constraints are met. Existing quantum compilation frameworks do not yet handle next-generation modular architectures, nor fault-tolerant (FT) circuits. In this work, we present the first hardware-aware compiler for mapping and routing FT circuits—spanning a wide range of topological QEC codes—onto chiplet-based superconducting architectures. By decoupling of high-level FT compilation from low-level hardware backends, we enable architectural retargetability, resulting in a significant reduction of compilation overhead. Compared to state-of-the-art compilers such as LightSABRE, our approach is more effective and scalable, delivering speedups of up to 20×, with an average decrease of 86.39% in circuit depth, and 91.4% in SWAP gate counts across multiple sizes of lattice surgery circuits. Our implementation delivers high-quality solutions, enabling the compilation of fault-tolerant circuits to next-generation modular chiplet architectures.

Contents

1 Introduction

Quantum Computing has the potential to solve complex problems in the near future, ranging from condensed matter physics to quantitative finance [1–3]. In contrast to algorithmic design, current quantum hardware is constrained by inherent physical noise, which limits both the depth and the width of executable circuits. This noise is primarily driven by environmental decoherence, which causes the loss of quantum information over time, and by imperfect gate operations, where interactions such as leakage lead to the occupation of non-computational states outside the qubit subspace. Furthermore, stochastic processes such as depolarization transform pure quantum states into mixed states, thereby reducing the possibility for successful computation [4].

To achieve sufficiently low error rates for executing algorithms with proven quantum advantage, specialized Quantum Error Correction (QEC) codes tailored to quantum systems are essential [5]. These codes encode a single logical qubit using multiple physical qubits. By distributing logical information, it is possible to suppress physical noise through redundant encoding. Because QEC codes introduce qubit overheads several orders of magnitude greater than uncorrected systems, it is necessary to develop large-scale quantum architectures that scale with these resource requirements [6]. To overcome the scalability limits of monolithic architectures, recent hardware designs have shifted toward modular architectures [7]. These architectures consist of smaller, interconnected quantum processors (QPUs) or chiplets that form a larger system [8].

Executing quantum algorithms requires constructing quantum circuits and compiling them to hardware backends to satisfy the underlying physical constraints, such as limited connectivity or defective qubits. Existing quantum compilation frameworks do not yet handle next-generation modular architectures, nor fault-tolerant (FT) circuits. FT circuits are inherently different from conventional quantum circuits, since operations are performed on logical qubits formed by combining physical qubits into spatial patches [9]. These patches possess a well-defined topological structure and physical requirements (distance d) that scale with the targeted error rate. For instance, achieving required error rates of 10^{-12} to support practical quantum algorithms necessitates a high code distance (e.g., $d \approx 27$ [5]). This translates to the overhead of hundreds of physical qubits for a single logical qubit [10]. FT circuits not only require the utilization of physical qubits for constructing logical qubits, but also additional physical qubits for realizing non-trivial gates [9]. Thus, circuit sizes are significantly larger. Since

scaling monolithic architectures to such vast resource requirements is limited by the diminishing fabrication yields, chiplet-based architectures have emerged as a viable alternative. These systems consist of multiple interconnected chiplets — small-scale, monolithic Quantum Processing Units (QPUs) connected using inter-chiplet links. Thus, the compilation process of next-generation compilers explicitly needs to incorporate hardware constraints and FT circuit structure, as well as scale to very large problem sizes. In addition to increased circuit complexity, future hardware architecture information must be considered. This is especially pronounced in chiplet-based architectures with limited connectivity between chiplets. These inter-chiplet links are notably slower and noisier than on-chip couplings. Excessive use of these noisy connections can introduce additional errors and limit the effectiveness of deployed QEC codes.

FT circuits consisting of logical qubits must be arranged carefully to preserve their inherent structure required for error correction. Mapping these patches onto physical hardware as regular circuits without considering both circuit and hardware layout results in a large number of additional operations (e.g., SWAP gates) and thus errors. If too many errors accumulate, the overall error rate can exceed the threshold at which QEC can suppress them, rendering it ineffective. Current approaches are generally divided into two distinct orientations: some incorporate hardware constraints but remain unaware of QEC's structural requirements (e.g., MECH [11], RouteForcing [12]), while others address QEC-specific needs (e.g., tQEC [13], QECC-Synth [14]) but fail to scale to the demands of FT circuits requiring modular chiplet architectures. Existing quantum compilation frameworks are largely unable to cope with the requirements of both chiplet-based architectures and FT circuits. To date, no approach has successfully combined both sets of requirements.

Given the described requirements, a potential solution must scale to a large number of qubits in a reasonable time, account for the well-structured nature of *QEC codes*, and incorporate hardware constraints imposed by a chiplet-based architecture. This results in mapping and routing operations on the circuit without rendering the inherent FT ineffective, and in increased modularity and flexibility during compilation. This leads to the main research question of this thesis:
How to *efficiently* map *QEC codes* on *chiplet quantum architectures*?

In this thesis, we present a compilation framework that bridges the gap between QEC codes and chiplet-based quantum architectures. The presented framework enables scalable mapping and routing of FT quantum circuits onto chiplet-based quantum architectures while preserving QEC structure and accounting for realistic hardware constraints. Furthermore, our approach decouples high-level FT compilation — such as lattice surgery — from low-level hardware backends. This abstraction ensures architectural retargetability and significantly reduces compilation overhead for frequently executed circuit. We design our approach around three core concepts: **(1) Partitioning**

of the logical circuit into a fixed number of partitions, as well as the extraction of their size. **(2) Chiplet-mapping** for assigning partitions to chiplets, whereas chiplets can hold multiple partitions based on the number of qubits required. **(3) Local and remote routing** to fulfill both local (intra-chip) and global (inter-chip) connectivity constraints of the fixed architecture, by inserting additional SWAP gates.

We integrate the proposed compilation approach into the Qiskit compilation framework. Our modular implementation is built on a pipeline of multiple compilation passes, enabling individual components to be seamlessly replaced or optimized. Furthermore, our custom chiplet backend abstracts the hardware layer, enabling easy configuration and evaluation of interconnected chiplet architectures.

To verify the suitability and efficiency of our approach, we evaluate our implementation across a range of realistic hardware configurations and constraints, benchmarking against SOTA algorithms [15]. Compared to SOTA algorithms, we reduce the two-qubit (2q) gate overhead and circuit depth overhead by *several orders of magnitude*, as well as *improved runtime* by up to $10\times$.

The core contributions of this thesis are:

- **Algorithmic design:** Development of a novel end-to-end framework for partitioning, mapping, and routing that integrates hypergraph-partitioning with bin-packing heuristics. This approach is specifically designed for the structural requirements of FT circuits and is aware of constraints imposed by modular, chiplet-based architectures.

- **Modular abstraction:** Decoupling of high-level FT compilation — such as lattice surgery — from low-level hardware backends, enabling architectural retargetability resulting in a significant reduction of compilation overhead for frequently executed circuits.

- **Compiler integration:** Development and integration of compilation passes into the Qiskit compilation framework [16].

- **Chiplet backends:** Development of a modular extension of QISKIT.BACKENDV2 for chiplet-based architectures. Implementation enables easy configuration of inter-chiplet links, definition of link noise models, configuration of defective qubits, reconfigurable on-chip long-range connections, and supports different qubit layouts.

- **Circuit construction:** Engineering of a circuit generation and conversion pipeline integrated with tqec [13], qiskit-qec [17], and Stim [18] by providing interoperability to support both QEC simulations and hardware execution.

- **Chiplet evaluation:** Evaluation of our implementation against existing compilers for FT circuits on chiplet-based architectures, specifically examining scalability and chiplet awareness.
- **QEC evaluation:** Evaluation of our implementation against a baseline approach to assess its impact on the resulting logical error rates. Furthermore, we quantify how various backend configurations influence the performance of FT circuits.

2 Background

2.1 Circuit Transpilation

There exist many different quantum compiler frameworks and software stacks, with the most common being PennyLane, Qiskit, and TKET [16, 19, 20]. While these frameworks may differ drastically from their low-level implementation and target hardware, they share a common general structure: given an input logical quantum circuit, generate a quantum circuit that satisfies all hardware constraints to prepare for execution on the target hardware backend [21].

2.1.1 Transpiler Workflow

The task of a quantum transpiler is the rewriting of a logical circuit in such a way that all underlying hardware constraints are fulfilled in order to ensure correct execution on the selected hardware backend [21]. Additional tasks, such as circuit optimization, are possible. The overview and workflow are shown in Figure 2.1.

The following list presents the most important compiler tasks, along with a short description for each. While the presented steps are taken from the Qiskit framework, the general task is shared by all available compiler frameworks [16]:

1. **Logical circuit optimization:** Optimization of the logical circuit, such as deletion of consecutive CNOT gates. Depending on the complexity of the optimization tasks performed, this stage can introduce significant runtime overhead.

2. **Gate decomposition:** Decomposition and rewriting of logical gates that cannot be executed on the target execution hardware.

3. **Qubit mapping:** Construction of a mapping of virtual qubits (from the initial logical circuit) to physical qubits (from target backend).

4. **Qubit routing:** Construction of routing paths that ensure that, whenever two qubits need to interact, they are placed in physical qubits that can interact with each other. Additional gates are added to the circuit to satisfy the connectivity constraints.

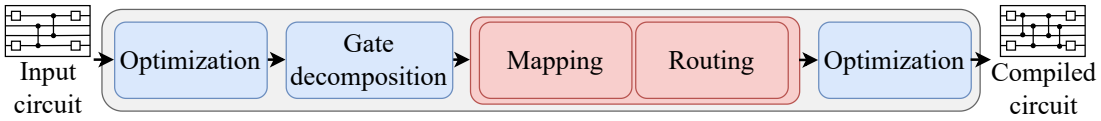

Figure 2.1: Overview of *Transpilation Stages* performed during circuit transpilation in order to compile a logical circuit to hardware for execution. Blue rectangles represent optional stages, whereas red rectangles must always be executed. While optimization is not strictly necessary, and gate decomposition is not always needed, mapping and routing must always be performed in order to account for all hardware constraints.

5. **Physical circuit optimization:** Optimization of the final circuit. While this is not always necessary, re-examining the circuit that has been previously changed by routing can further improve the final circuit.

2.2 Hardware Technologies

In the following section, the current and most established quantum computing platforms are briefly introduced. The landscape of quantum hardware is diverse, ranging from electromagnetic traps to solid-state circuits. **Trapped-ion** platforms utilize charged particles as qubits that are stored/trapped in electromagnetic fields [22]. In contrast, *neutral atom* platforms trap uncharged ions using optical lattices or tweezers [23]. **Photonic** quantum computing utilizes single photons as qubits either on-chip or using free-space optics [24]. Similarly, **superconducting** architectures utilize an on-chip setup. They are engineered using microwave-frequency electronic circuits, in which qubits are implemented as nonlinear oscillators, e.g., via Josephson junctions [25]. Many other approaches exist, such as NV-center-based or spin-based platforms [26].

To meet the high resource demands of future quantum systems, recent architectures have moved away from traditional monolithic single-chip designs: *Chiplet* and *distributed* architectures [7]. *Chiplet* architectures connect multiple smaller chips using inter-chiplet links (over which quantum gates can be executed) to form a QPU [27], as shown in Figure 2.2. This poses the advantage that high-fidelity local operations and existing fabrication techniques can still be utilized. *Distributed* (DQC) architectures consist of multiple QPUs (also commonly referred to as cores) connected via either classical channels or links capable of sharing purely quantum information (e.g., to distribute pairs of entangled qubits). Since, in general, it is assumed that QPUs could be located at geographically separate sites, remote inter-core communication is more expensive in both time and noise than local (inter-chiplet) communication.

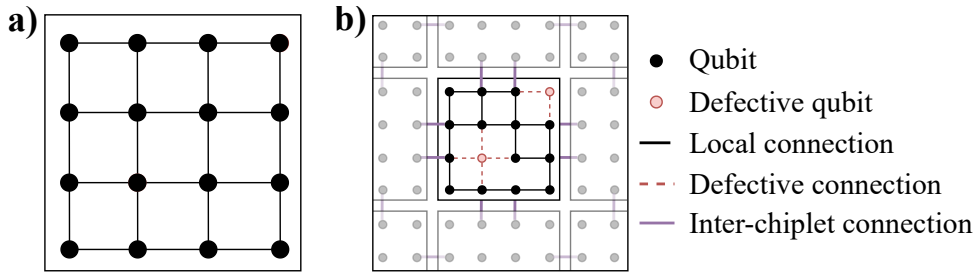

Figure 2.2: Overview of different backend architectures. ***a)*** *monolithic backend and* ***b)*** *quantum chiplet backend. The architecture consists of a square grid of interconnected quantum chiplets. Each chiplet features a square qubit lattice, with adjacent chiplets connected via inter-chiplet links. Defective qubits can compromise both local and inter-chiplet connectivity.*

2.3 Quantum Error Correction

Quantum Error Correction (QEC) is the quantum version of classical error correction tailored to the hardware and systems requirements of quantum computers.

Unlike classical error correction, it is not possible to perform any measurements on the physical qubits used, since this would destroy all information stored. To circumvent this issue, so-called ancilla qubits are utilized. In order to interact with a logical qubit, syndrome measurements are performed on these ancilla qubits [28].

As long as physical errors are below the threshold value, increasing the number of physical qubits can exponentially suppress errors on the logical qubits. The minimum required physical error is often denoted as the threshold value.

When evaluating the strength of a QEC code, three evaluation criteria are often utilized [29]: **(1)** What types of errors can be corrected? To correct for all possible errors, a QEC code must be able to correct both bit- and phase-flip errors. **(2)** What is the encoding rate? The encoding rate describes how many logical qubits can be encoded given a fixed amount of physical qubits. **(3)** What logical gates are natively supported? To perform logical operations, it is necessary to support logical gates with minimal overhead.

2.3.1 Stages

To ensure that no errors occur and that errors can be detected and corrected, the following overview in Figure 2.3 visualizes the most important steps necessary for quantum error correction to work.

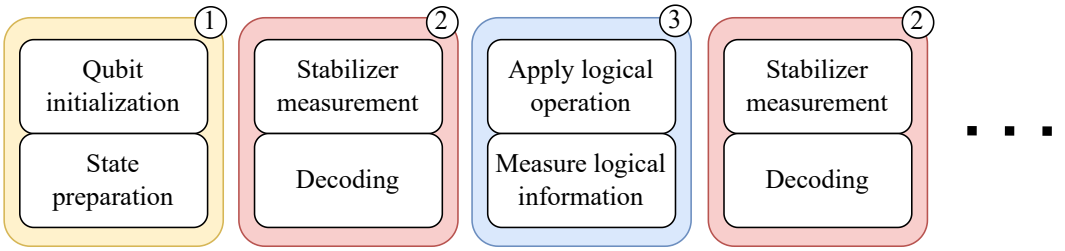

Figure 2.3: Overview of the general procedure of quantum error correction [29]. **(1)** In the first step, a QEC code is selected, and a general logical state is initialized. **(2)** To stabilize the logical state, the correction step measures out the stabilizer and decodes it into syndromes. **(3)** Logical operations and measurements can then be performed, followed immediately by the correction stage. Subsequently, this sequence is repeated.

1. **State preparation:** Physical qubits are initialized and projective measurements into the codespace are performed.

2. **Stabilizer measurements:** Ancilla qubits are prepared and measured in such a way that the qubits in the state preparation stage are not modified. The measurements from this stage are often called syndromes and passed to the decoding stage. In the absence of errors, the syndrome will have the same value in every round. In case a syndrome changes, a local error occurred. Global errors, on the other hand, cannot be detected, since these are indistinguishable from logical operations.

3. **Decoding:** The task of the decoder is to detect changes in the syndrome and compute a prediction which qubits are the source of the change. While many decoding algorithms exist, they are generally limited to a subset of codes or incur high runtime complexity to achieve favorable results. Additionally, higher decoder efficiency results in a lower logical error rate. There is often a tradeoff between runtime and decoder efficiency.

4. **Correcting for errors:** If the decoder detected some errors, these need to be corrected. It is possible to add gates to the circuit to fix the error; however, this introduces additional errors, so it is generally not performed. Instead, a table containing all errors that affect the measured results, based on known errors, is updated.

5. **Apply logical operations:** In case we have a stable system with no or known errors, it is possible to perform logical operations between logical qubits. Following

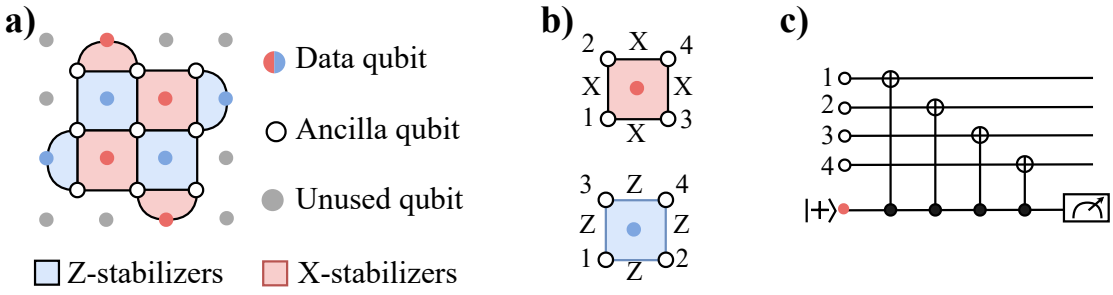

Figure 2.4: Overview of a surface code patch with the corresponding syndrome measurement circuit. ***a)*** *Overview of a distance d = 3 surface code patch on a rotated square lattice.* ***b)*** *Colored rectangles define X (red) and Z (blue) plaquette stabilizers operating on ancilla and data qubits.* ***c)*** *Circuit definition of a X-type plaquette stabilizer circuit. For a Z-type plaquette, the control and target of the CZ gates are flipped.*

this, decoding and correction are performed in order to ensure no errors have occurred.

2.3.2 QEC Codes

One of the most widely studied classes of error correction codes is that of [[n, k, d]] stabilizer codes. The numbers n, k, and d are used to characterize the physical requirements of the code [28, 29].

- **n:** Minimum number of physical qubits required for code construction (excluding required ancilla qubits)
- **k:** Number of logical qubits spanned by the code
- **d:** The code distance is the measure of strength of the code, and is the length of the smallest undetectable error chain—that is, the length of the smallest logical operator. To account for measurement errors, it is necessary to perform d rounds of error correction to fully account for all errors.

While there are many other classes of error-correcting codes, we will limit our focus to stabilizer codes, as these can *efficiently* be simulated but require some additional non-Clifford gates. While there exist some approaches for the realization of such non-Clifford gates for QEC codes, these require non-trivial constructs and are further described in Section 2.3.3.

Surface Code. One of the most studied and well-established QEC codes is the surface code [30], which encodes logical qubits into a 2D regular square patch of entangled physical qubits [9]. There exist two versions: the original planar surface code and the improved rotated surface code, which reduces the required physical qubits by approximately half. The latter is the most widely deployed version [31].
Quantum LDPC code. A new class of QEC codes capable of significantly reducing the resource requirement of the surface code is that of *Quantum Low Density Parity Check (qLDPC) Codes* [32]. A family of stabilizer codes is an LDPC code if every check acts on a constant number of qubits, and every qubit is involved in a constant number of checks.

While it is possible to reduce the qubit requirement by up to one magnitude over the surface code, specialized hardware requirements are necessary: non-local and long-range qubit connectivity [10]. This stems from the code formulation, which uses a weight-4 nearest-neighbor lattice with two additional long-range couplers per qubit, resulting in a total of 6 connections. The long-range couplers result from the embedding of a torus on a planar surface, thus leading to periodic boundaries.

While small qLDPC codes have been experimentally verified, future scalable approaches still face major challenges [33].

2.3.3 Logical Operations for QEC Codes

While stabilizer codes can be efficiently simulated and, to some extent, easily implemented, they are not capable of universal quantum computation. To define a universal set for quantum computation, most stabilizer codes use the (Clifford + T) gate set. The *T gate* is a special case of a non-Clifford gate, often realized either using magic state cultivation, distillation, or code switching [34]. Clifford gates can be realized using lattice surgery [9]
Lattice Surgery. Lattice surgery provides a framework for performing a general set of elementary logical operations that can be performed fault-tolerantly on qubits encoded using the surface code [36, 37]. An example for lattice surgery implementing a logical CNOT gate is given in Figure 2.5. The lattice surgery formulation can be extended to a generalized framework, opening the possibility of operating on qubits encoded with qLDPC codes [10].
Magic state cultivation and distillation. To implement a universal gate set, most stabilizer codes use the (Clifford + T) gate set. The Eastin-Knill theorem [38] forbids any single code from having a transversal implementation of all gates needed for universality. Thus, the T gate is implemented using either magic state. Magic states are specially prepared ancilla qubits used in the *state injection process* to enable non-Clifford-like effects on logical qubits. Depending on the available resources and qubit

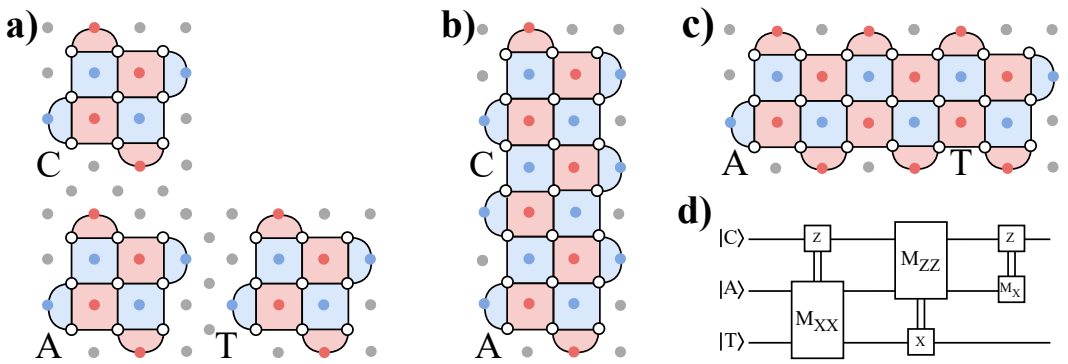

Figure 2.5: Overview of a CNOT operation realized using lattice surgery. ***(a)*** *Visualization of three memory patches necessary to realize the lattice surgery CNOT gate on logical qubits* ***C****ontrol and* ***T****arget, utilizing an* ***A****ncilla patch. In addition to one ancilla patch, multiple ancilla qubits are necessary to perform merge and split operations between patches.* ***(b) - (c)*** *Visualization of merging operation in order to perform logical measurements on the respective logical patches.* ***(d)*** *Circuit description of the lattice surgery CNOT operation [35].*

noise, it is possible to implement magic states using either *cultivation* or *distillation* [39, 40]. Distillation *purifies* a small batch of *imperfect* magic states, generated by a *factory*, into fewer, but higher-fidelity, magic states [40]. Cultivation, on the other hand, prepares small, fault-tolerant magic states and increases the code distance until reaching a desired logical error rate [39].

2.3.4 Simulating QEC

In general, stabilizer codes can efficiently be simulated using existing simulators, such as Stim and Qiskit [16, 18]. To simulate QEC experiments, it is necessary to simulate stabilizer circuits and decode measured syndrome measurements. For this, a multitude of open-source libraries exist, each with different features and drawbacks: limited support for noise channels (e.g., amplitude decay) and support for Clifford gates only [18, 41].

Simulator. Stim [18] is one of the most widely adopted high-performance simulators of stabilizer QEC circuits. It enables the simulation of large-scale circuits with up to 100k qubits. Libraries such as CUDA-Q [41], as well as Qiskit [16], provide similar capabilities.

Decoder. During the execution of a QEC cycle, it is necessary to use an efficient and effective decoding algorithm. The *Minimum weight perfect matching (MWPM)* algorithm

is the most popular decoding algorithm for surface codes and is implemented in the PyMatching library [42]. For qLDPC codes *Belief Propagation-Ordered Statistics Decoding (BP-OSD)* decoders are the current state-of-the-art, but suffer from higher decoding latency than surface code decoders [43]

3 Overview

This chapter outlines the system overview and the general workflow developed for this thesis. While recent advancements in chiplet-based quantum architectures and QEC are accelerating the transition towards the post-NISQ era, existing compilers struggle with the resulting circuit complexity and hardware requirements. This gap motivates the main research question of this thesis:

> How to *efficiently* map *QEC codes* on *chiplet quantum architectures*?

To address this, we propose a compiler consisting of specialized mapping and routing routines tailored to FT circuits for chiplet-based architectures. In addition, we further refine our design objectives and propose solutions to bridge the gap between QEC requirements and architectural limitations.

The main goals of our proposed compiler are to: (1) maintain the structure of code patches, (2) adapt to defective qubits, and (3) account for the constraints introduced by inter-chiplet links. To maintain the code path structure, a standard quantum circuit representation is no longer sufficient, as it lacks critical structural information such as code patch descriptions. This stems from the fundamental structural differences between FT and conventional circuits.

Our approach is designed to keep high-level FT compilers, such as lattice surgery, independent from low-level backends. By decoupling these steps, we significantly reduce overall compilation time, since it is no longer necessary to re-run high-level compilation when switching between backends. Furthermore, this allows simultaneous compilation of multiple FT circuits to a single backend. In addition, our proposed approach addresses critical hardware constraints, specifically inter-chiplet noise and defective qubits.

Our proposed approach is fully integrated into the Qiskit transpiler, enabling seamless use within existing workflows and easy comparison, allowing us to validate if our proposed approach meets the defined goals. We evaluate the implementation across various circuits and backends — including different chiplet configurations and inter-chiplet link constraints — and compare the results against SOTA algorithms, such as SABRE.

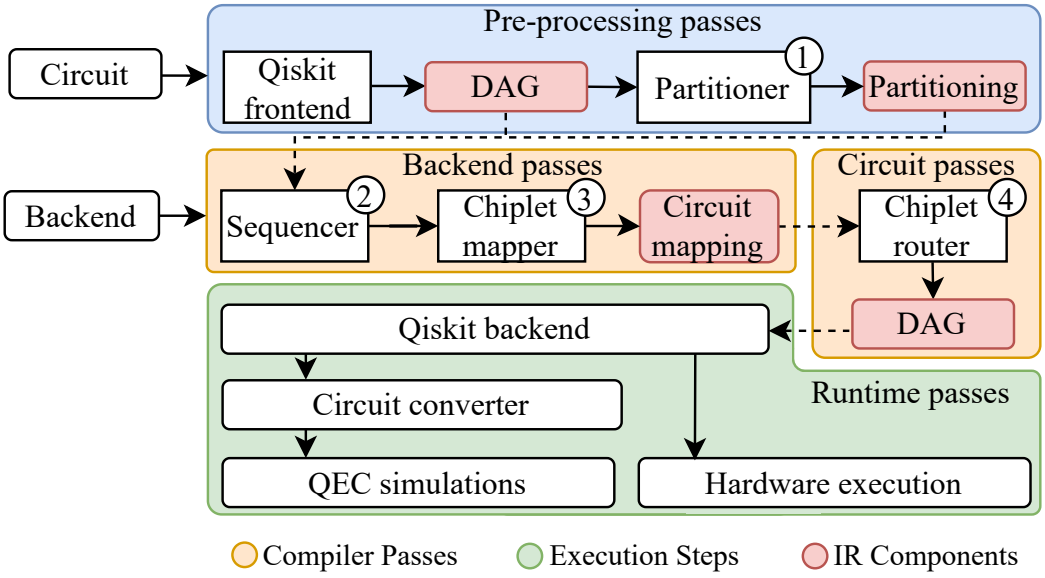

Figure 3.1: Overview of system design of our approach presented in this thesis. Our modular approach integrates with Qiskit and supports circuit generation for both hardware execution and simulation. We divide the system into three levels: the front-end is implemented via a preprocessing pass. The middle-end is implemented via backend and circuit passes. This is followed by the backend, which consists of several runtime passes, enabling both simulation and hardware execution. IR components, marked in red, can be viewed and evaluated during compilation.

3.1 Compiler Architecture

Figure 3.1 visualizes major ideas and the general approach implemented in this thesis. As shown, we present both the general idea and the method used at each step to realize the approach outlined in the initial idea.

To address the initial research question, the proposed approach comprises three main components. Before the workflow is started, it is necessary to provide a quantum circuit to map and the backend on which it should be mapped. According to this specification, **(1)** the provided quantum circuit is partitioned into a fixed number of partitions based on its structure. **(2) - (3)** After partitioning, the extracted partitions are ordered and then mapped to the specified backend, consisting of either one monolithic chip or multiple connected chiplets. **(4)** To fulfill all connectivity constraints, the mapped partitions are routed as the last step, inserting additional SWAP gates to satisfy local connectivity

constraints. We extend on this description in Section 3.2 with an algorithmic and in-depth explanation.

3.2 System Components

In the following section, we will introduce the main constituents of the system design. We split the system into four distinct passes, each representing a different level of operation. While the preprocessing pass represents a general frontend, both the backend and circuit passes represent a mid-end containing different circuit IRs. The final runtime pass represents the backend, enabling both execution and simulation.

An in-depth algorithmic description of all system components and their implementation is given in Section 4.

3.2.1 Pre-Processing Passes

To modify the input circuit, it must be converted to a suitable format. We utilize the *Qiskit frontend* to support a wide range of input formats (e.g., OpenQASM [44]) and circuit types (e.g., dynamic circuits) to generate a *Directed Acyclic Graph* (DAG). After this conversion, we also construct a hypergraph from the quantum circuit. This is utilized in the *Partitioner*, which generates a circuit partitioning for the constructed hypergraph. The partitioning and DAG are passed on to the subsequent backend pass.

3.2.2 Backend Passes

The backend pass receives the constructed DAG and the circuit's partition description, along with a backend configuration. The backend configuration describes the physical hardware architecture. Prior to performing any mapping, a *Sequencer* generates an ordering of the circuit partitions. Both the partition ordering and the backend configuration are passed to the *Chiplet mapper*, which generates a mapping from partitions to chiplets. In addition, logical circuit qubits in partitions are assigned to physical qubits on the backend.

3.2.3 Circuit Passes

Given the initial DAG and the mapping of circuit qubits to physical qubits, it is necessary to construct a DAG that satisfies all constraints imposed by the initial DAG and the hardware architecture. A *chiplet router* is introduced to construct a new DAG and insert additional operations. While this increases the circuit's complexity and depth, the DAG remains logically equivalent. This router is suitable for monolithic

hardware backends, but it also includes specialized functionality to support chiplet-based backend architectures. Following this step, the resulting DAG description of the circuit is passed to a subsequent pass.

3.2.4 Runtime Passes

Given a DAG that satisfies all logical and hardware constraints, it is passed to the *Qiskit backend*. This allows us to support a wide range of additional tasks (e.g., optimization, error mitigation [45]) and interoperability with other compiler frameworks [44, 46]. After completing optional backend tasks, it is possible to either perform QEC simulations or execute the circuit on real hardware. Before performing simulations, it is necessary to convert the circuit into a valid QEC simulation format (e.g., a Stim circuit [18]).

3.3 Technical Challenges

Due to the problem's complexity, we briefly outline the rationale for our design — a framework guided by theoretical constraints and optimized using heuristic approaches.

3.3.1 Circuit Partitioning

When a circuit exceeds the qubit resources of a single monolithic chip, it can be distributed across a chiplet-based architecture. While chiplets enable scalability, this distribution comes at the expense of using expensive inter-chiplet links in addition to on-chip connections. Consequently, the circuit must be grouped into a set of partitions. Since finding an optimal distribution is an NP-hard problem [47], we employ a heuristic *Partitioner* to generate efficient partitions that minimize inter-chiplet communication.

3.3.2 Partition Mapping

Mapping partitions onto chiplets is an NP-hard optimization problem [48]. To manage this complexity, we employ a heuristic approach to the mapping process, consisting of two stages: **(1)** A *Sequencer* constructs an order for processing partitions. **(2)** A *Mapper* assigns these partitions to chiplets based on the generated order.

Furthermore, the Mapper is designed to be patch size-aware. This flexibility is critical, as the system must accommodate varying sizes of different QEC codes — such as the geometries of the Surface [30] and QLDPC codes [10].

3.3.3 Partition Routing

Qubit allocation and routing are known to be NP-complete problems [49]. To address this, a *Chiplet Router* utilizes a two-stage routing pass composed of **(1)** local and **(2)** inter-chiplet routing. Unlike established approaches such as SABRE [15], our method does not include an additional partition-mapping phase during routing. Consequently, the DAG remains unchanged. Once it passes through the Chiplet Router, it undergoes no further modifications.

4 Design

The following chapter provides a detailed description of the proposed approach. We will introduce the high-level approach and specify the backend definition in Section 4.1. An in-depth algorithmic description of each compilation pass in the high-level approach is provided in Section 4.3.

4.1 Overview

We present the core architecture of our approach as a four-stage pipeline that compiles FT circuits to chiplet architectures. In Section 4.3.1, the input circuit is converted into a viable description. Section 4.3.2 details the partitioning stage, where circuit-qubits are partitioned into discrete groups. Section 4.3.3 then describes the mapping strategy for distributing these partitions across multiple chiplets. Finally, Section 4.3.4 describes the routing procedure and specifically addresses how our approach integrates inter-chiplet links into the pathfinding process.

4.1.1 Backend Definition

We consider a chiplet architecture C as a set $\{c_1, c_2, \ldots, c_N\}$, with $N = 2^k$ as system size. We define c_i as an individual monolithic chiplet connected to its neighbors in a rectangular grid. We define n_{icc} number of inter-chiplet links ICC as a list of tuples $(q_{icc,i}, \epsilon)$. We define $q_{icc,i}$ as the local qubit q_i supporting at most one inter-chiplet link to a neighboring chiplet with error rate ϵ. A qubit q_i can exist in one of two states: *functional* or *defective*. If q_i is in the defective state, then the set of all incident connections is similarly mapped to the defective state.

4.2 Workflow

To make a circuit executable on hardware, it must be mapped and routed. We define the *mapping* step as an operation that determines a mapping $\pi : v_i \rightarrow q_i$ assigning each virtual qubit to a corresponding physical qubit in the backend, with v_i being a virtual qubit of the original quantum circuit. The routing step adds *SWAP* gates between qubits that need to interact but cannot because of their fixed connectivity.

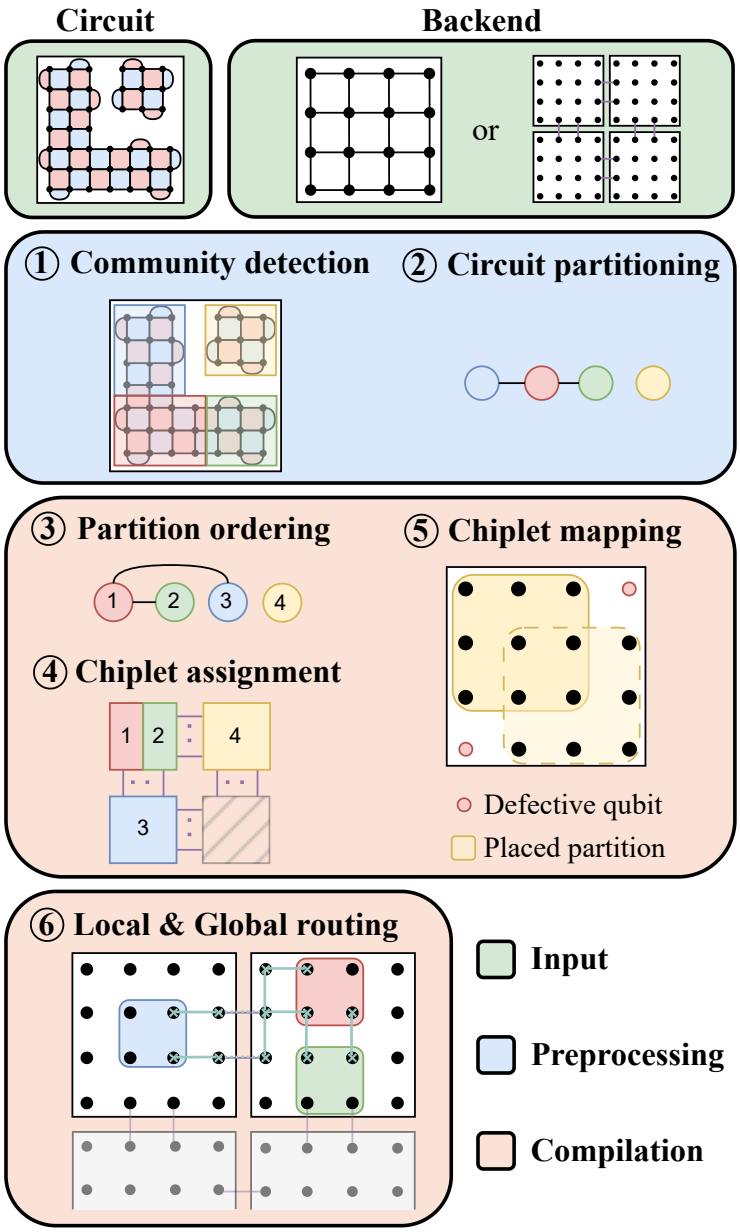

Figure 4.1: Detailed workflow given by a step-by-step example showing the approach taken in this thesis. An FT circuit can be compiled to either a monolithic or a chiplet backend.

We briefly explain our implementation. We split the mapping step into two parts.

- **Partitioning** of the logical circuit into a fixed number of partitions, as well as extracting their size.
- **Chiplet-Mapping** then assigns every partition to a chiplet, whereas chiplets can hold multiple partitions based on number of qubits required.
- ***Local* and *remote* routing** in order to fulfill both local (intra-chip) and global (inter-chip) connectivity constraints of the fixed architecture. To meet local constraints, we implement a standard SWAP routing routine. Global routing uses a custom cost-aware routing algorithm that accounts for inter-chiplet connections.

After performing the aforementioned steps, we have a mapping of virtual to physical qubits as well as a circuit that fulfills all hardware constraints. Figure 4.1 illustrates the complete workflow of the proposed approach.

4.3 Compilation Passes

4.3.1 Pre-Processing

During pre-processing, we convert a quantum circuit into a hypergraph and a DAG, which are required by partitioning algorithms. We construct the hypergraph by mapping qubits to nodes and representing edges based on the interactions between virtual qubits. The hypergraph is implemented using a compressed sparse row (CSR) format, consisting of an edge list and a pointer vector.

Following this, the input circuit is converted into a DAG. The DAG formalism is common in compilers and enables easier transformations.

4.3.2 Partitioning

The partitioning pass constructs a partition of the initial hypergraph. We formally define the partitioning problem as a mapping of virtual qubits from the initial circuit into k partitions p using a partition function

$$\pi : v_i \rightarrow p_j \quad \text{with } j \in [0, k] \tag{4.1}$$

Given the inherent patch structure of fault-tolerant circuits, their partitioning comes naturally. Since patches in FT circuits are defined to require no additional routing overhead, we do not partition the circuit into as many partitions as available chiplets [50], but rather into the number of error-correcting patches present in the circuit. To

estimate the number of present patches, we employ a two-stage approach to determine the optimal number of partitions based on the circuit, not the hardware. During the first stage, we employ a community detection algorithm [51] to roughly estimate the number of partitions. In the second step, we partition the circuit using a hypergraph partitioner [52].

We employ this approach for simple circuits, whereas for large-scale FT circuits, we use predefined partitions supplied by higher-level compilers. Such compilers are commonly used for computing fault-tolerant circuits from logical circuits.

4.3.3 Mapping

Following the partitioning stage, we map virtual circuit qubits to physical qubits on the specified backend. We employ a three-step approach comprising the calculation of *(1) partition ordering*, *(2) partition-to-chiplet assignment*, and *(3) chiplet mapping* as shown in Algorithm 1.

(1) Partition ordering. To assign partitions to chiplets, we require a partition ordering that specifies the order in which partitions are considered. We implement PartitionOrdering in Algorithm 1. We construct a partition ordering using the *BFS* (breadth-first search) algorithm applied to an entanglement graph. The entanglement graph is constructed from partitions as nodes and models the interaction between partitions using the interactions of the virtual qubits. We iteratively visit all nodes of the entanglement graph and perform BFS from each unvisited node. We group partitions into sets, each containing partitions reachable from any other via existing dependency edges in the entanglement graph. Given this disjoint set of partitions, we can construct partition orderings for multiple circuits simultaneously.

(2) Chiplet assignment. Given the calculated partition order, we implement ChipletAssignment in Algorithm 1 using a greedy bin-packing problem (BPP) algorithm to assign and place partitions on chiplets. We reformulate the partition assignment problem as a 2D BPP [53], where chiplets act as bins containing rectangular regions. We implement two variants of the first-fit algorithm [54] for solving the 2D BPP, differing in the placement of the first partition on a chip bin: *center* places it in the center of the selected chiplet. *size_aware* employs a top-left approach based on the bottom-left heuristic [53], where the partition is placed at the top-left as far as possible. We implement this approach in PlacePartition in Algorithm 1. After assigning a partition to a chiplet bin, we remove the corresponding region and perform a recursive guillotine cut [55] to recalculate free regions. The process is shown in Figure 4.2. For partitions that interact with each other, we implement a tight placement approach PlacePartitionRelative in Algorithm 1. Our implementation selects the nearest available region on the same chip bin as the reference partition. If no free region is available, we pick the closest chip

Algorithm 1 Mapping and placement of partitions onto chiplets

```
procedure MAPPING(C, P)
    P_ordered ← CALCULATEPARTITIONORDERING(P)
    π_C ← CHIPLETASSIGNMENT(C, P_ordered)
    QubitMapping ← CHIPLETMAPPING(C, π_P)
    return QubitMapping
procedure PARTITIONORDERING(P)
    Initialize S_visit = ∀p ∈ P
    C_components, C_visited ← []
    for p ∈ S_visit do                          ▷ Iterate over disjunct set of components
        if p ∈ C_visited then
            skip                                ▷ Skip visited partitions
        C_visited.APPEND(p)                     ▷ Perform BFS from this node
        C_partitions ← list of p.NEIGHBOURS in BFS-order
        C_components.APPEND(C_partitions)       ▷ Append all connected nodes
    return C_components
procedure CHIPLETASSIGNMENT(C, P_ordered)
    π_C ← []                                    ▷ Mapping of partition to chiplet
    for C_partitions ∈ P_ordered do
        p_relative ← ∅
        for p ∈ C_partitions do                 ▷ Iteration given partition order
            if p_relative = ∅ then              ▷ Place first partition on backend
                π_C ← C.PLACEPARTITION(p)
                p_relative = p
            else                                ▷ Place partition relative to prior
                π_C ← C.PLACEPARTITIONRELATIVE(p, p_relative)
    return π_C
procedure CHIPLETMAPPING(C, π_C)
    m_v,q ← []                                  ▷ Virtual to physical qubit mapping
    for c ∈ C do                                ▷ Iterate over chiplets
        for p ∈ π_c do                          ▷ Iterate over partitions on chiplet
            vq ← C.MAPPARTITION(p)              ▷ Assign physical qubits to partition
            m_v,q.APPEND(vq)
    return m_v,q
```

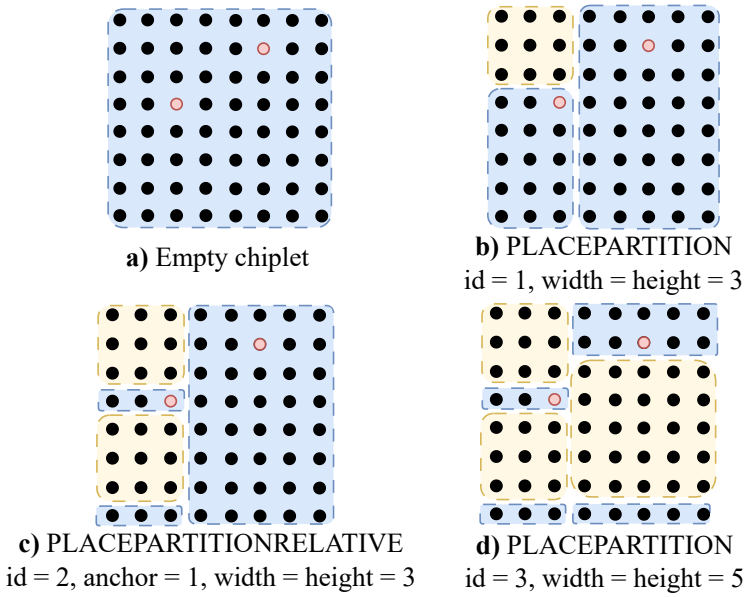

Figure 4.2: Visualization of placement approach in Algorithm 1. ***(a) - (d)*** *Placement of partitions on a chiplet containing two defective qubits using* PlacePartition *and* PlacePartitionRelative *utilized in Algorithm 1. The size_aware variant for first-partition placement is used. The id field serves as the unique identifier for the new partition, and the anchor defines the existing partition relative to which it is positioned.*

bin that can fit the current partition. We extract the relative orientation of partitions from the labels of the virtual qubits in the logical circuit. By comparing these labels, we can determine whether a partition should be placed below or to the right of the relative partition.

To ensure that the mapping phase is aware of defective qubits, we extend the chiplet bins with *no-placement zones*. These zones represent the locations of the defective qubits that cannot be used and are accounted for in the BPP.

Our implementation follows the general heuristic approach commonly applied in the literature [12, 56]: Assign as many partitions as possible to each chiplet to minimize inter-chiplet communication between partitions on different chiplets.

(3) Chiplet mapping. Following the calculation of partition to chiplet assignment and placement, we calculate a placement mapping using ChipletMapping in Algorithm 1. We construct the placement mapping using a dictionary that assigns virtual qubits to physical qubits by iterating over all chiplets and partitions assigned.

4.3.4 Routing

Following the mapping of virtual to physical qubits, we need to ensure that, whenever two qubits interact in the initial circuit, they are placed in physical qubits that can interact with each other. We fix the calculated qubit mapping and construct a new quantum circuit with additional SWAP operations between distant physical qubits to satisfy connectivity constraints.

We employ a two-step approach during routing: (1) local routing on chiplets and (2) global routing across chiplets implemented in Routing in Algorithm 2.

Local routing. We implement a greedy shortest-path routing approach LocalRouting in Algorithm 2 for qubits on the same chiplet. Given the construction of FT circuits, patches are designed to avoid any additional routing. Thus, we ignore qubits of the same partition. For intra-chiplet partition interactions, we employ SWAP routing along the shortest path. This path is bifurcated, and routing is simultaneously performed from both qubits to the midpoint. We calculate the shortest unweighted path using Dijkstra's algorithm [57].

Global routing. We implement a chiplet-aware routing in RemoteCostRouting in Algorithm 2. To take hardware constraints into consideration, we formulate a total cost function consisting of three interdependent components derived from the most important hardware-aware metrics:

$$\textsc{PathCost}(P, q_{icc}) = \underbrace{|P|}_{\text{Path length}} + \underbrace{\alpha \cdot ICC(q_{icc})}_{\text{Inter-chiplet cost}} + \underbrace{\beta \cdot U(q_{icc})}_{\text{Congestion penalty}} \tag{4.2}$$

With $D(q_i, q_j)$ as the shortest path between q_i and q_j, $ICC_k(c_i, c_j)$ the noise level of the k-th inter-chiplet link between chiplets c_i and c_j, and $U_k(c_i, c_j)$ the number of utilizations of the inter-chiplet link. α and β represent tunable hyperparameters:

- Varying α prioritizes selecting the inter-chiplet link with the highest fidelity
- Varying β prioritizes selecting the path that utilizes the least congested inter-chiplet link.

Given an initial *shortest* path between qubits of different chiplets, we select the inter-chiplet link resulting in the lowest path cost according to 4.2. We identify the k-nearest inter-chiplet links adjacent to the current selection. After calculating the total cost for each, the path with the minimum cost is selected, and the corresponding inter-chiplet link utilization counter is incremented to reflect its usage. By varying the parameters α and β in Equation 4.2, we enable fine-grained control over the routing focus, balancing competing metrics such as code distance reduction and increased circuit depth. The optimal values of α and β depend heavily on the specific circuit structure and require fine-tuning based on the inter-chiplet noise.

Algorithm 2 *Local* and *remote* routing for chiplet based backends

```
1: procedure ROUTING(C, P)
2:     icc_u ← []
3:     for (q_c, q_t) ∈ Two-Qubit gates do
4:         if q_c and q_t ∈ qubits(C) then                ▷ Routing for local gates
5:             LOCALROUTING(Circuit, q_c, q_t)
6:         if q_c and q_t ∉ qubits(C) then                ▷ Routing for remote gates
7:             icc ← REMOTECOSTROUTING(Circuit, q_c, q_t)
8:             icc_u.APPEND(icc)                          ▷ Update inter-chiplet utilization
9: procedure LOCALROUTING(Circuit, g, q_c, q_t)
10:    P ← DIJKSTRA-PATH(q_c, q_t)
11:    for (q_1, q_2) ∈ P do
12:        CIRCUIT.ADD(SWAP, q_1, q_2)
13:    CIRCUIT.ADD(G)                                     ▷ Add gate g
14: procedure REMOTECOSTROUTING(Circuit, q_c, q_t, α, β)
15:    ICC ← INTER-CHIPLETCONNECTIONS(C_{q_c})            ▷ Calculate
16:    P_paths, P_cost ← []
17:    for q_icc ∈ ICC do                                 ▷ Find optimal inter-chiplet connection
18:        P_shortest ← DIJKSTRA-PATH(q_c, q_icc)
19:                   + DIJKSTRA-PATH(q_icc, q_t)
20:        P_cost ← PATHCOST(P_shortest, q_icc)           ▷ Defined in Equation 4.2
21:        P_paths.APPEND(P_shortest)
22:        P_cost.APPEND(P_cost)
23:    P_optimal ← min_{p∈P_cost} P_path
24:    for (q_1, q_2) ∈ P_optimal do                      ▷ Routing along selected path
25:        CIRCUIT.ADD(SWAP, q_1, q_2)
26:    CIRCUIT.ADD(G)
27:    return icc                                         ▷ Utilized inter-chiplet connection
```

5 Implementation

The following chapter provides an overview and description of integrating our approach into a full compiler by utilizing the Qiskit software stack [16]. Section 5.1 provides a brief overview of the implementation in the Qiskit transpiler. Section 5.2 describes the integration, whereas Section 5.3 describes the necessary extensions that we developed.

5.1 Overview

We integrate the presented approach into Qiskit v2.30rc1 [16] in Python v3.12 by constructing a custom pass manager pipeline using the available qiskit.StagedPassManager instance. We use networkX [58] in combination with the KaHyPar framework [52] for circuit partitioning. Our global routing implementation utilizes Dijkstra's algorithm from rustworkx [59].

To generate FT circuits, we use the TQEC [13] library, which can generate circuits based on the surface code and lattice surgery. We use Stim [18] and PyMatching [42] for performing QEC simulations. The simulations employ a modified *SI1000* error model [60], a superconducting-inspired circuit error model [61], that properly handles inter-chiplet links. All circuits are converted to the Stim format [18] using the qiskit-qec [17] library. To support lattice surgery, we extended the circuit conversion in qiskit-qec.

5.2 Qiskit Integration

In order to utilize current and future features in Qiskit [16] in Python v3.12, as well as to simplify comparison to existing SOTA approaches, we integrate our *mapping and routing* approach as a custom StagedPassManager plugin using best practices from the Qiskit documentation [16]. The constructed pipeline consists of three qiskit.PassManager instances implementing the *init*, *layout*, and *routing* stages. The *init* stage consists of an qiskit.AnalysisPass for translating the input circuit into a hypergraph. The *layout* and *routing* stage consists of multiple qiskit.TransformationPass instances implementing our circuit partitioning, mapping, and routing approaches. Through the use of the existing generic interface classes, seamless integration into the transpiler pipeline and existing transpiler stages is possible. Every pass can then be either a *analysis* or a

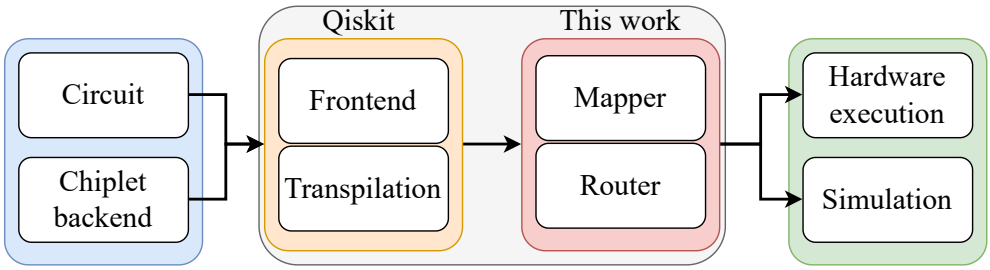

Figure 5.1: Broad overview of our approach integrated into the Qiskit compiler framework. An input circuit and a backend are fed into the Qiskit frontend and transpilation pipeline. The presented approach then compiles a circuit that can be used for hardware execution or simulation.

transformation pass. While an analysis pass (in our case, the partitioning step) does not change the circuit, a transformation pass (in our case, the mapping and routing steps) modifies the circuit itself.

5.3 Construction of a Chiplet Backend

In order to map circuits to a chiplet-based backend, it was necessary to extend Qiskit with a chiplet backend based upon the pre-existing *GenericBackendV2* class. We implement the extension using the GenericChipletBackendV2 class. This class contains all the properties and methods required for a backend to be used with the Qiskit transpiler.

The developed backend can construct various monolithic and chiplet architectures, as summarized in Figure 5.2.

5.3.1 Chiplet Specific Backend Properties

In the following, we extend the pre-existing backend with chiplet-specifiec properties in order to fully define a generic chiplet backend. In addition to the specified new properties, it was necessary to add two additional properties that are not defined by the abstract GenericBackendV2: (i) remote connections and (ii) defective qubits.

- **Qubit topology:** Qubit arrangement on a chip. Can be one of the following: *grid* or *rotated_grid*. Grid implements a simple qubit grid arrangement. rotated_grid shifts every second row and slightly changes the connectivity.
- **Qubit connectivity:** Local qubit connectivity. Can be one of the following: *nn* or *toric*. The *nn* option constrains local qubits to nearest neighbor connectivity, while

toric adds two additional long-range connections.

- **Chiplet topology:** Physical arrangement of chiplet. Currently, only the *grid* arrangement is supported, which lays out chiplets in a 2d grid or rectangle (depending on the number of utilized chiplets).
- **Chiplet connectivity:** Number of remote connections between every chiplet. At least one connection must be shared between every chiplet. A disjoint setup, where certain parts of the device can not be reached, is currently not supported.
- **Chiplet connectivity noise**: Float value determining noise level of every remote connection.
- **Chiplet connectivity noise type**: Type of noise for determining the noise level of remote connections. Can be one of the following: *constant* or *randomized*. *Constant* noise assigns the same noise level to all remote connections. *Randomized* noise assigns a randomized noise to all remote connections (further described below).
- **Chiplet connectivity noise amplification**: Scalar value in order to perform noise amplification in the case of randomized noise assignment to remote connections.
- **Device size:** Tuple (c1, c2) specifying number of chiplets in x- and y-direction (in the case of *grid* arrangement).
- **Chiplet size:** Tuple (n, m) specifying number of qubits in x- and y-direction on every chiplet.
- **Number defective qubits:** Scalar value determining the number of defective qubits on every chip.

5.3.2 Inter-Chiplet Links

Inter-chiplet links are defined using existing *ECR* gates, which are then used to extend the backend with a list of tuples of the form *(remote connection id, remote noise)*. This list is used both during the *remote-routing* process and error-model construction for simulations.

The noise level e of a remote connection between node i, j is computed utilizing a randomization function r for generating a random noise factor in the range $[1, 10]$

$$r = \min(\max(1,\ u \cdot 10)\,,\ 10)\,, \quad \text{where } u \sim \mathcal{U}(0,1)$$

$$e_{i,j} = \min(0.9,\ r \cdot \text{noise\_amplification} \cdot \text{noise})$$

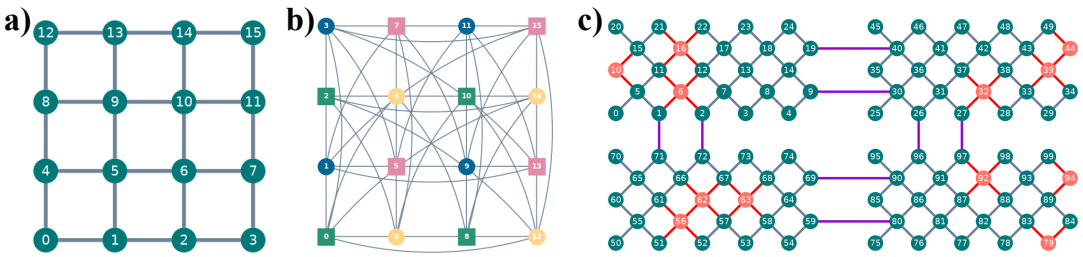

Figure 5.2: Overview of available backend architecture configurations. ***(a)-(b)*** *monolithic and* ***(c)*** *chiplet backend configurations with varying degrees of local connectivity. Inter-chiplet links between chiplets are represented in violet, whereas red connections and qubits are defective.*

5.3.3 Defective Qubits

To implement defective qubits, the backend uses a second coupling map: (i) a fully functional coupling map with all qubits and connections, used to extract the general structure of each chiplet. (ii) A defective coupling map, on which defective qubits and connections are removed.

During partitioning and mapping, the fully functional coupling map is used to construct the chiplet bins. The definition of the *no-placement zones* uses the location of defective qubits extracted from the defective coupling map. In addition, the defective coupling map is used during the routing passes.

6 Evaluation

6.1 Overview

To assess our implementation and conduct a thorough evaluation, we structured the evaluation into three stages. Section 6.2 details the first stage, describing the evaluation environment, including libraries and benchmark circuits utilized. Second, Section 6.3 assesses the applicability and scalability of existing solutions on chiplet backends. Section 6.4 goes beyond existing solutions and evaluates our implementation with SOTA approaches.

6.2 Evaluation Setup

To perform a thorough evaluation of our presented approach, it is necessary to assess existing approaches. Since many steps during the evaluation depend on the usage of existing libraries, we provide a short description of the steps and requirements for setting up an evaluation experiment.

Implementation: We integrate our implementation into Qiskit v2.30rc1 using Python v3.12. We use Stim [18] with 10^8 shots per simulation run for QEC simulations, and PyMatching [42] for decoding. All experiments are performed on one AMD EPYC 7713P 64-Core Processor with 1 TiB of main memory.

Benchmarks: We utilize Qiskit [16] and tQEC [13] to generate circuits for simulations used in the evaluation.

Metrics: To assess the performance of our implementation, we utilize the following metrics: (1) **Gate Overhead:** Number of additional two-qubit gates added. (2) **Depth Overhead:** Increase of circuit depth (critical path length through the circuit). (3) **Logical Error Rate:** Simulated logical error rate of the circuit prior to and after running the compilation steps.

Baseline: We utilize the LightSABRE [15] algorithm, implemented in Qiskit, as our baseline algorithm. It is a well-established algorithm known for its runtime efficiency and production of state-of-the-art results. Its prevalence in the literature ensures good comparability with existing results.

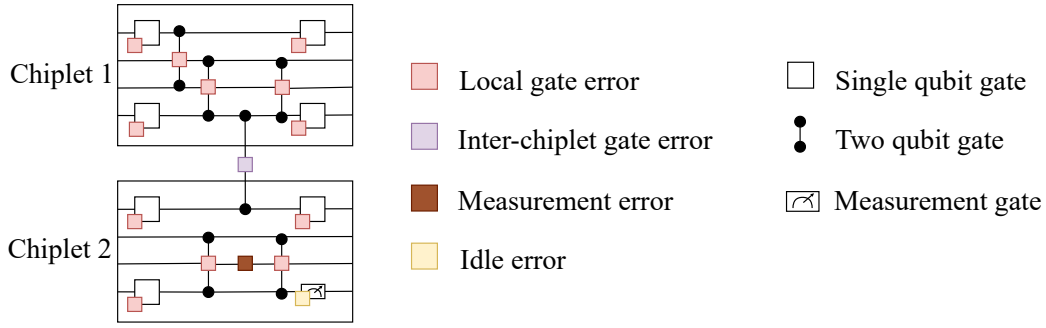

Figure 6.1: Extended *SI1000* error model introducing logical, inter-chiplet, idle, and measurement errors to operations on and between chiplets *Chiplet 1* and *Chiplet 2*.

6.2.1 FT Circuit Generation

We implement multiple circuits used for simulation and evaluation: a generic *CAT* circuit consisting of distributed CAT states, distributed *memory circuits* consisting of surface code patches, and a (distributed) *lattice surgery CNOT* circuit. While the first circuit can be generated natively in Qiskit, the latter memory circuits are generated using tQEC [13].

As simulations with Stim require a specialized *Stim circuit representation* [18], we use qiskit-qec [17] to perform Qiskit-to-Stim and Stim-to-Qiskit conversions. We extend the conversion utilities in qiskit-qec to convert circuits that perform multiple QEC cycles, since this was not yet possible.

6.2.2 Circuit-Level Noise Model

To simulate realistic quantum systems, it is necessary to introduce errors into the circuits. A number of different error models exist, such as *uniform_depolarizing*, *si1000*, or *EM3*. These error models try to simulate expected errors in different systems. We use the *SI1000* noise model, a superconducting-inspired circuit error model [61]. We extend this noise model to account for inter-chiplet links by introducing a specific gate error for these gates. An overview of errors and their definition is given in Figure 6.1.

6.3 Existing Approaches

In the following section, we evaluate existing solutions to determine a baseline for further evaluation and comparison with our approach.

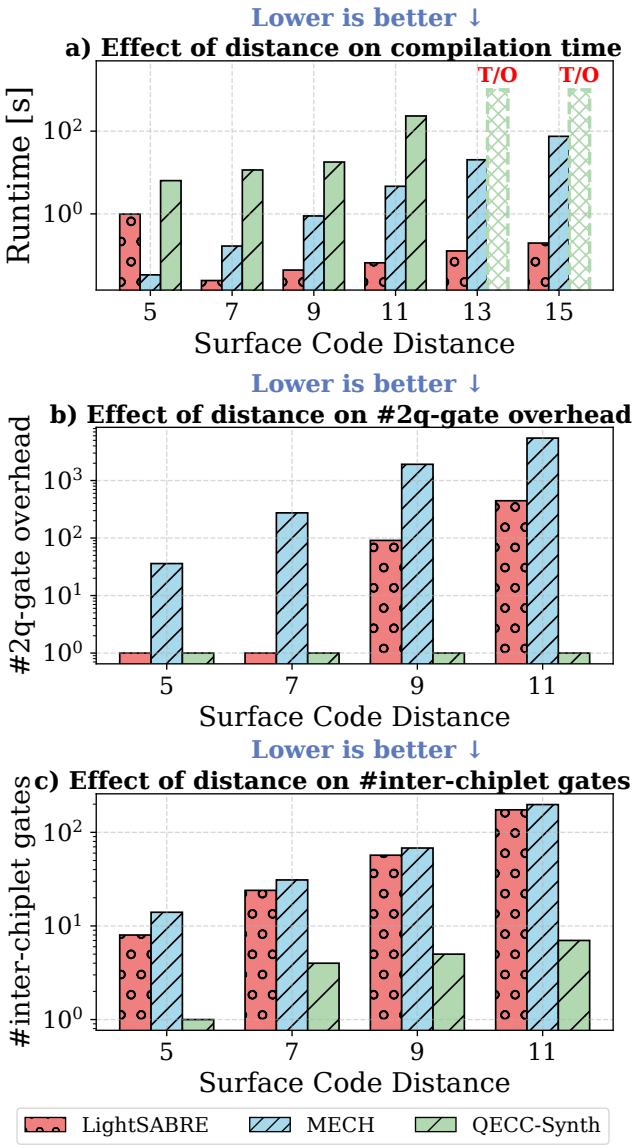

Figure 6.2: Comparison of compiling a single surface code memory patch by general-purpose (LightSABRE [15]), chiplet-dedicated (MECH [11]), and QEC-aware (QECC-Synth [14]) compilers. ***(a)*** *Compilation runtime with runs exceeding* 10^3 *s reported as timeouts (T/O).* ***(b)*** *Two-qubit gate overhead introduced when compiling to a monolithic backend.* ***(c)*** *Number of two-qubit gates executed over inter-chiplet links when compiling to a chiplet backend.*

In the following evaluation, we utilize existing approaches, each tailored to specific constraints, for mapping and routing a surface code memory patch on a chiplet backend of varying size. As our standard baseline, the LightSABRE algorithm [15] is selected. MECH [11] serves as the baseline algorithm for chiplet approaches. QECC-Synth [14] serves as the baseline algorithm for QEC approaches.

In Figure 6.2(a), we evaluate the scalability of a general-purpose mapper (LightSABRE [15]), a chiplet-dedicated compiler (MECH [11]), and a QEC-aware approach (QECC-Synth [14]) across circuits constructed from surface-code patches [9] with increasing code distance, and thus increasing circuit size, targeting a monolithic backend.

Figure 6.2(b) evaluates the scalability effect by reporting the two-qubit gate overhead (e.g., SWAPs) introduced by general-purpose and specialized compilers when mapping QEC code patches onto a monolithic grid backend.

In Figure 6.2(c), we evaluate how the compilers perform mapping and routing of the surface code patch on a chiplet-based architecture by measuring the number of two-qubit gates executed over inter-chiplet links.

Given the results achieved by current SOTA approaches in Figure 6.2, the LightSABRE [15] algorithm, natively implemented in Qiskit, is selected as the baseline algorithm for the following section.

6.4 Implementation Evaluation

Section 6.4.1 assesses the scalability of our compiler, focusing on execution runtime and circuit statistics to validate its scalability on large-scale circuits. In Section 6.4.2, we investigate the performance of our implementation on chiplet-based architectures, by considering varying numbers of inter-chiplet links, random noise, and the presence of defective qubits. Finally, Section 6.4.3 presents the results of QEC experiments running with and without compilation, in order to assess the impact of the compilation process. We structure our evaluation around distinct research questions targeting the quality of the compiled circuit.

6.4.1 End-to-End Performance

The following two research questions assess the scalability of our framework, focusing on execution runtime and circuit statistics to validate its scalability on large-scale circuits.

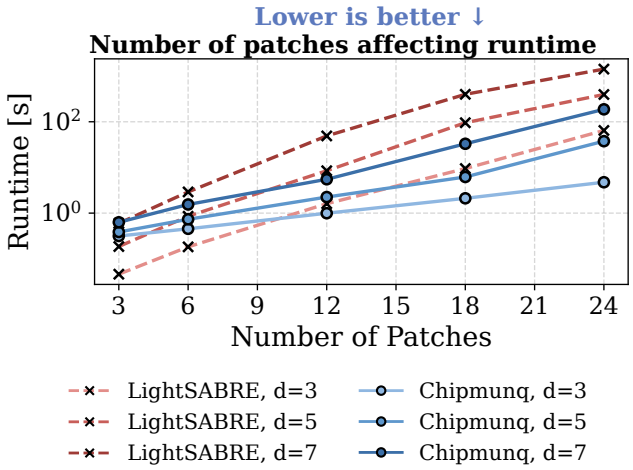

Figure 6.3: Runtime comparison between LightSABRE and our implementation (Chipmunq) for a varying number of QEC patches and surface code distances.

RQ#1: *How does the runtime scale as circuit size increases?*

Hypothesis. In general, we expect the runtime and performance to scale (near) linearly with circuit size for our implementation. For the baseline algorithm, we expect superlinear scalability due to its greater complexity.

Methodology. In order to determine the scaling behavior, we utilize a CNOT circuit constructed from three patches using lattice surgery. To evaluate performance, we scale the circuit size by duplicating the CNOT. In order to show the scalability for more complex circuits, we generate the circuit for different code distances in the range $d = [3, 5, 7]$.

Setup. We are going to use **(1)** the described CNOT circuit, as this shows the patch detection, mapping, and routing implementation of our algorithm. **(2)** To show the scalability, we create a number of copies of this CNOT circuit. **(3)** The circuit will be mapped to a backend consisting of three times the number of chiplets as patches, to ensure, with near certainty, that enough qubit resources are available. Every chiplet in the backend is the same size as each patch of the circuit, ensuring that each patch is assigned to exactly one chiplet.

Takeaway. Our implementation demonstrates significant performance gains, outperforming the LightSABRE algorithm by up to $10\times$ for medium-scale circuits. As illustrated by the scalability curves in Figure 6.3, our approach incurs lower overhead, making it better suited to the increasing demands of large-scale circuits.

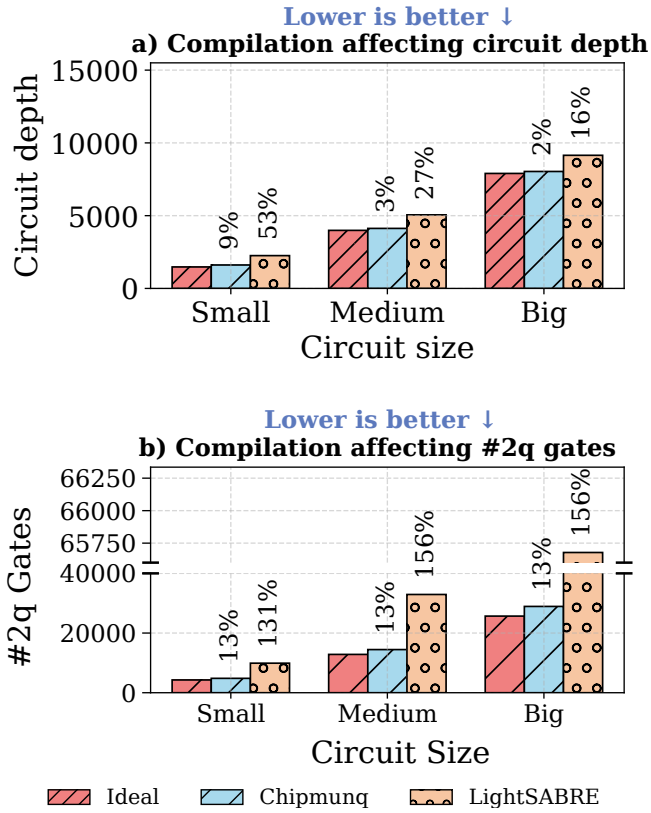

Figure 6.4: Comparison of LightSABRE and our implementation with the initial circuit (Ideal) with regards to circuit statistics for varying circuit sizes when compiled to a chiplet backend. ***(a)*** *Resulting circuit size and* ***(b)*** *circuit depth resulting from compilation. The utilized circuit consists of 3, 9, and 18 patches for the Small, Medium, and Big circuits, respectively. Ideal represents the initial circuit without any compilation.*

RQ#2: *How do circuit depth and gate overhead scale as circuit size increases?*

Hypothesis. We expect our implementation to only slightly increase the circuit depth and gate overhead, since keeping the patches together drastically decreases overhead.
Methodology. Similar to RQ#1, we utilize a CNOT circuit constructed using lattice surgery for $d = 3$.
Setup. **(1)** We utilize the described CNOT circuit. **(2)** We construct three circuit cases: *small*, *medium*, and *big*. Each circuit consists of $[1, 3, 6]$ CNOT circuits, respectively. **(3)** The circuit will be mapped to a backend consisting of three times the number of chiplets

as patches, to ensure, with near certainty, that enough qubit resources are available. Every chiplet in the backend is the same size as each code patch, ensuring that each patch is assigned to a distinct chiplet.
Takeaway. Our implementation demonstrates a significant reduction in circuit depth and two-qubit gate overhead as compared to LightSABRE, capable of reducing the two-qubit gate overhead by up to $30\times$. Our findings are supported by improved results across varying circuit sizes, as shown in Figure 6.4.

6.4.2 Chiplet Suitability

The following two research questions investigate the performance of our implementation on chiplet-based architectures, by considering varying numbers of inter-chiplet links, random noise, and the presence of defective qubits.

RQ#3: *How does the number of inter-chiplet links influence circuit routing?*

Hypothesis. Given a limited number of inter-chiplet links, we expect the depth of the routed circuit to go up drastically, as we expect a high congestion at these links. Depending on the routing, this can heavily influence the resulting circuit depth and gate overhead.
Methodology. Similar to RQ#1, we utilize a CNOT circuit constructed using lattice surgery for $d = 3$ and map this circuit, consisting of 3 patches, to a backend with a varying number of inter-chiplet links.
Setup. **(1)** We utilize the described CNOT circuit. **(2)** We construct three backend cases: *full*, *half*, and *limited*. Each backend consists of 8, 4, and 1 inter-chiplet links between all connected chiplets, respectively. **(3)** In order to show the importance of our custom routing method, we run the experiment for two inter-chiplet noise levels $[1e-4, 1e-2]$ with random noise enabled on all inter-chiplet links. **(4)** In order to evaluate the impact of reducing the inter-chiplet links, we calculate the 2q gate overhead and circuit depth overhead over the default (non-compiled) circuit.
Takeaway. Our implementation is aware of specific hardware constraints as shown in Figure 6.5 for experiments with varying the number of inter-chiplet links and their assigned noise values. Depending on the number of available inter-chiplet links, our implementation prioritizes high-fidelity links, accepting increased circuit depth as a trade-off. This behavior can be observed in Figure 6.5.

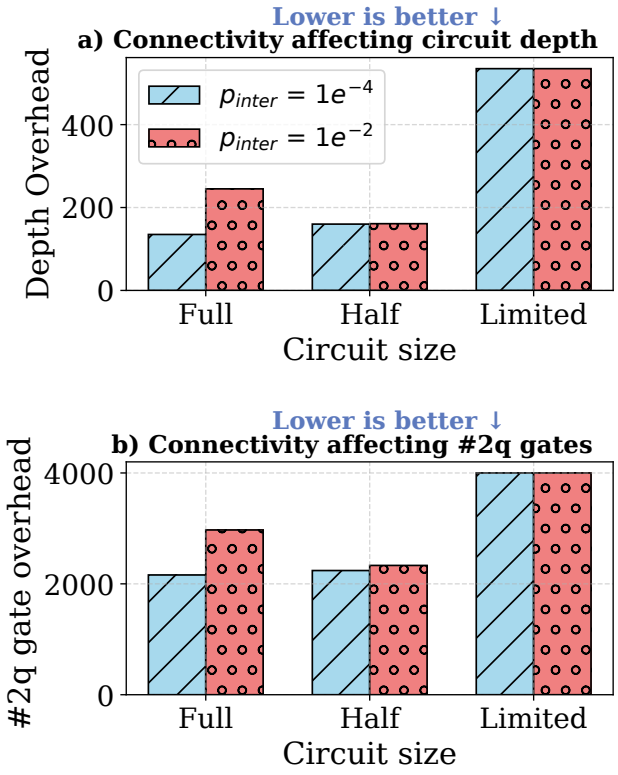

Figure 6.5: Effect of inter-chiplet noise levels on circuit statistics. ***(a)*** *Circuit depth overhead and* ***(b)*** *two-qubit gate overhead of the compiled circuits to a chiplet backend consisting of a high (10^{-2}) and low (10^{-4}) random inter-chiplet noise assignments. Utilization of the implemented cost routing method results in a higher circuit depth and overhead, resulting from the selection of high-fidelity inter-chiplet links.*

RQ#4: *How do defective qubits affect the resulting circuit?*

Hypothesis. Our implementation is aware of qubits that can not be used for mapping and routing. The mapping and routing will become significantly more difficult, and the resulting circuit will most likely have a much higher depth. The higher the number of unusable/defective qubits, the worse the overhead of circuit depth and backend utilization.

Methodology. Similar to RQ#1, we utilize a CNOT circuit constructed using lattice surgery for $d = 3$ and map this circuit, consisting of 3 patches, to a backend with a varying number of defective qubits.

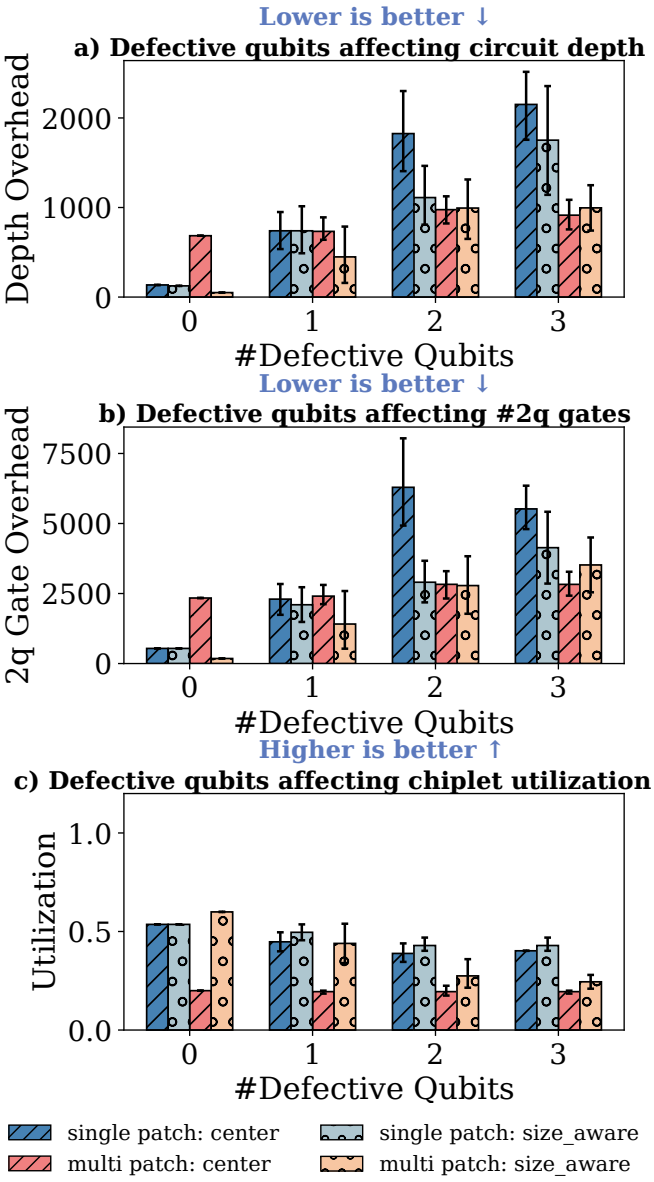

Figure 6.6: Effect of defective qubits on circuit and backend statistics. ***(a) - (c)*** *Influence of defective qubits on the effectiveness of our implementation for varying numbers of defective qubits per chiplet. We compare the two available placement methods center and size_aware, and evaluate each backend 10 times for different defective qubit placements.*

Setup. **(1)** We utilize the described CNOT circuit. **(2)** We compile the circuit to a fully connected chiplet backend with two sizes: *single_patch*, where one chiplet can fit one patch (plus some additional spacing), and *multi_patch*, where one chiplet can fit multiple patches. During evaluation, we vary the number of defective qubits and utilize two mapping routines: while *center* places a patch at the center of the assigned chiplet, *size_aware* places a patch at an optimized location (based on patch and chiplet size. **(3)** After compilation, we calculate circuit depth overhead, 2q gate overhead, and backend utilization in order to determine the influence of defective qubits. The backend utilization is defined in Equation 6.1 with a higher utilization corresponding to a more space-efficient compilation.

$$\text{Backend Utilization} = \frac{N_{\text{qubits}}}{N_{\text{qubits per chiplet}} \cdot N_{\text{chiplets}}} \in (0, 1] \tag{6.1}$$

Takeaway. Our implementation efficiently handles defective qubits on chiplets. By supporting an improved mapping routine, it is possible to achieve higher QPU utilization than with a naive implementation. Results are presented in Figure 6.6.

6.4.3 QEC Suitability

The following three research questions evaluate different QEC experiments, running with and without compilation, to assess the impact of the compilation process and backend configurations.

RQ#5: *How does the logical error rate of lattice surgery operations change when distributed to multiple chiplets?*

Hypothesis. We expect the resulting logical error rate of the compiled circuits to be worse than the non-compiled version, due to increased routing resulting from inter-chiplet links. This is furthermore influenced by the physical error rate of the inter-chiplet links.
Methodology. Similar to RQ#1, we utilize a CNOT circuit constructed using lattice surgery for code distances $d = [3, 5, 7]$ on a chiplet backend with full connectivity between chiplets.
Setup. **(1)** We utilize the described CNOT circuit. **(2)** We compile the circuit to a fully connected chiplet backend, where each chiplet fits one patch. **(3)** We evaluate the influence of the chiplet distribution by varying the noise level of the inter-chiplet links in the range $[10^{-4}, 10^{-3}, 10^{-2}]$. The resulting logical error rate of the compiled circuit is then used to quantify the influence when distributing a circuit to multiple chiplets.

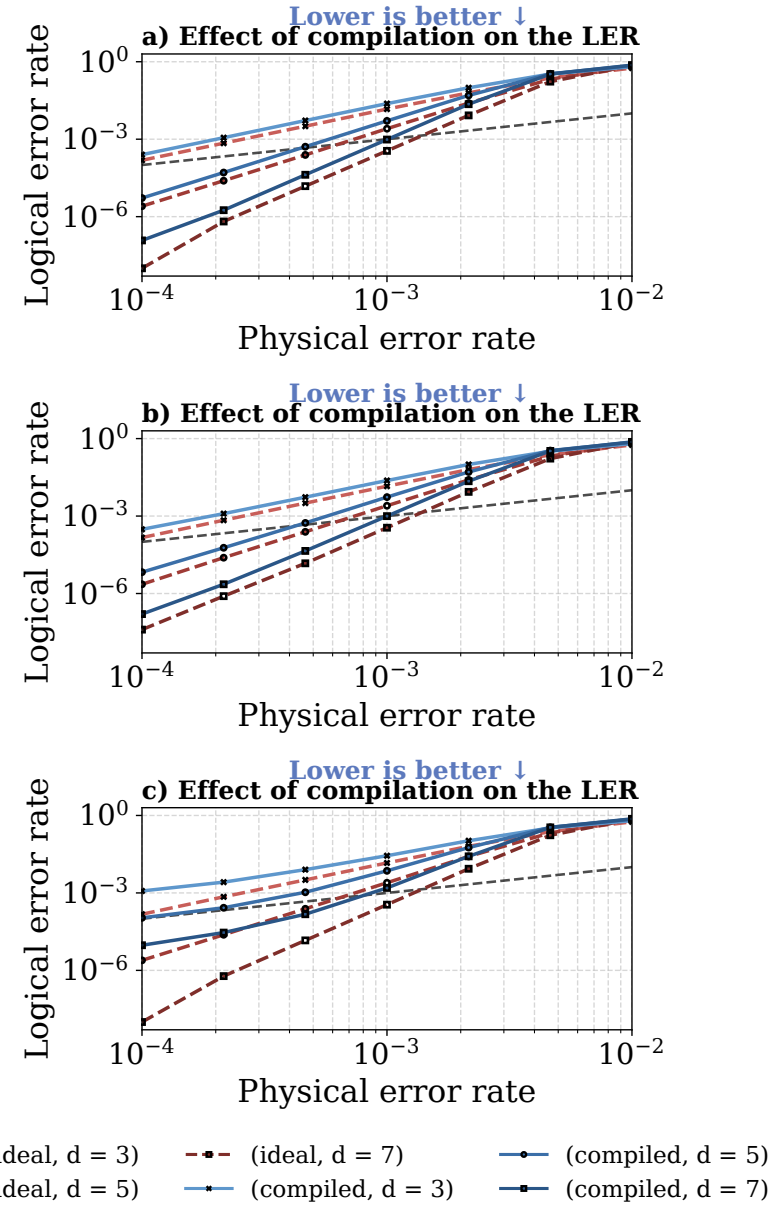

Figure 6.7: Effect of patch distribution on logical error rate of the CNOT lattice surgery operation. ***(a) - (c)*** *Influence of the compilation process on the LER for compiled and ideal circuits with varying code distances. Utilized inter-chiplet noise levels:* $10^{-4}, 10^{-3}, 10^{-2}$ *(top to bottom).*

Takeaway. Our implementation is capable of compiling QEC circuits to chiplet backends for varying inter-chiplet noise rates. While the logical error rate increases after compilation, it is still possible to exponentially suppress the physical error rate. The logical error rate is generally increased by up to one order of magnitude over a *default* (non-compiled) version, as visible in Figure 6.7.

RQ#6: *How is the logical error rate influenced by a limited number of inter-chiplet links?*

Hypothesis. We expect that for sparse inter-chiplet links, the logical error rate is increased drastically, as the resulting circuit depth is increased. We expect a plateau where additional inter-chiplet links do not further change the resulting logical error rate. This will probably depend strongly on the code distance used in the circuit; if there are fewer inter-chiplet links than the code distance, we expect a drastic increase in the logical error rate. For more inter-chiplet links than the code distance, we do not expect any influence.
Methodology. Similar to RQ#1, we utilize a CNOT circuit constructed using lattice surgery for code distances $d = [5, 7]$ on a chiplet backend with a varying number of inter-chiplet links between chiplets.
Setup. **(1)** We utilize the described CNOT circuit. **(2)** We compile the circuit to a chiplet backend with a varying number of inter-chiplet links: $[8, 6, 4, 2, 1]$. **(3)** We evaluate the influence of varying the number of inter-chiplet links by comparing the logical error rate of inter-chiplet connectivity constrained compilations with the logical error rate of the full-connectivity backend. Additionally, we set the noise level of the inter-chiplet links to 10^{-3}.
Takeaway. By limiting the number of available inter-chiplet links, the resulting logical error rate can be increased drastically. Results indicate that reducing the number of inter-chiplet links has a minimal impact on the logical error rate, provided the number of links remains above the code distance. However, once connectivity falls below this threshold, the logical error rate increases heavily, suggesting that code distance acts as a critical bottleneck. Figure 6.8 visualizes the increase of the logical error rate for varying numbers of inter-chiplet links and code distances. In addition, the reduction factor relative to the full connectivity version is shown.

RQ#7: *How is the logical error rate influenced by utilizing different routing algorithms?*

Hypothesis. We expect that, depending on the noise configuration of inter-chiplet links, it is possible to outperform the basic routing method by focusing on more complicated metrics than simply reducing the routing path.

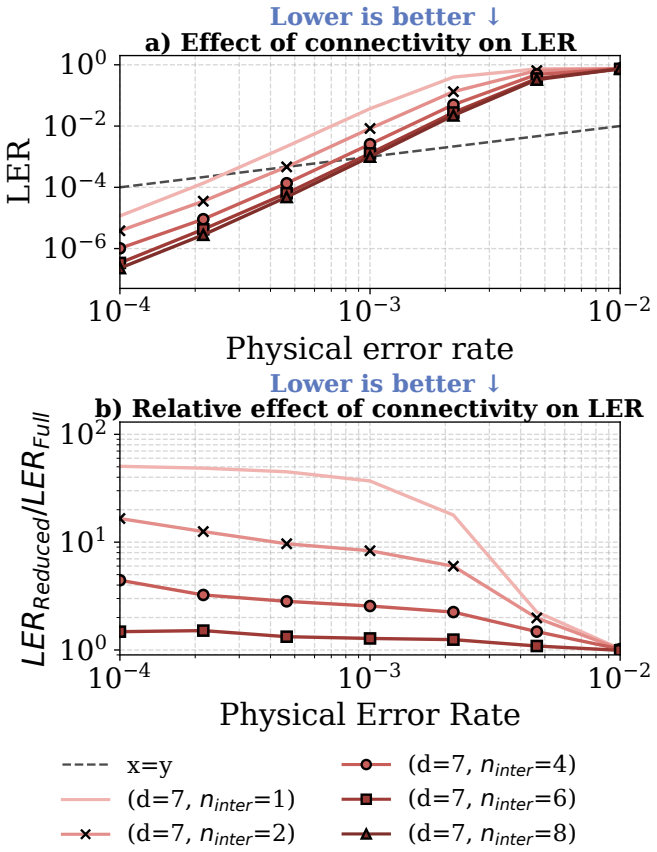

Figure 6.8: Influence of constraining the inter-chiplet connectivity between chiplets on the resulting logical error rate for inter-chiplet noise level 10^{-3}. ***(a)*** *Resulting logical error rate when compiling a circuit to backends with a varying number of inter-chiplet links.* ***(b)*** *Increase of the logical error rate for connectivity-constrained backends over the full-connectivity backend.*

Methodology. Similar to RQ#1, we utilize a CNOT circuit constructed using lattice surgery for code distances $d = 5$ on a chiplet backend with full connectivity between chiplets with different inter-chiplet noise configurations and routing methods during compilation.

Setup. **(1)** We utilize the described CNOT circuit. **(2)** We compile the circuit into a fully connected chiplet backend, where each chiplet fits one patch. For the inter-chiplet links, we select the random noise model and define two noise level configurations: *low variance* scales the noise randomly in the range $[1, 10]$, while *high variance* scales the noise randomly in the range $[1, 100]$. **(3)** During the compilation, we experiment with

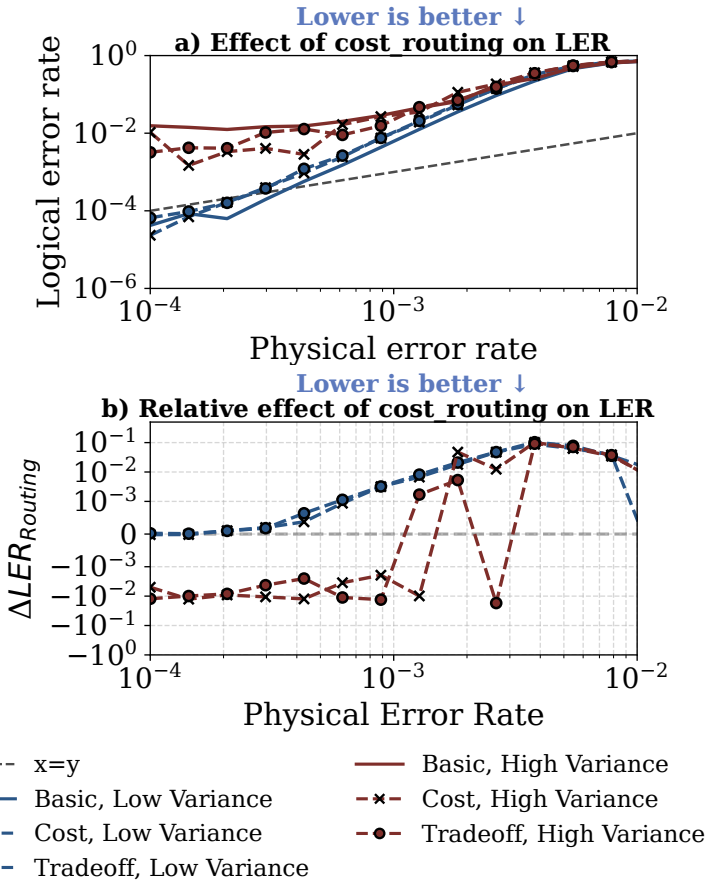

Figure 6.9: Influence of different routing methods on the resulting logical error rate for inter-chiplet error of 10^{-3}. ***(a)*** *Resulting logical error rate when compiling a circuit to a chiplet backend using different routing methods and considering different inter-chiplet noise configurations.* ***(b)*** *Improvement (or worsening) of the logical error rate by using either the cost or tradeoff routing method over the basic routing method.*

the utilization of three different routing methods: *basic* routing method does not take the inter-chiplet links noise into account; the *cost* routing method selects the inter-chiplet links with the lowest error on the chip; the *tradeoff* routing method calculates a tradeoff between choosing the inter-chiplet links with the lowest noise and the shortest path. **(4)** For evaluating the influence of selecting different metrics during routing, we compare the resulting logical error rate of the compiled circuits, similar to *RQ#6*.
Takeaway. By integrating a novel cost metric during the routing phase, our imple-

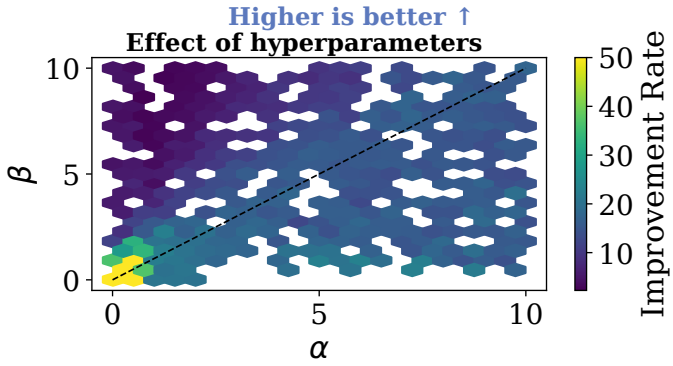

Figure 6.10: Influence of cost routing hyperparamters for inter-chiplet error of 10^{-3}. *Improvement rates over the basic routing method simulated for 1000 runs, varying α and β within the range of* $[0, 10]$.

mentation effectively mitigates the impact of high error variance across inter-chiplet links. As shown in Figure 6.9, this noise-aware approach outperforms the naive routing strategy, achieving a superior logical error rate in heterogeneous noise environments. This performance gain remains consistent across a wide range of hyperparameter configurations, as illustrated in Figure 6.10.

7 Related Work

This chapter reviews existing approaches for solving the mapping and routing problem, focusing on solutions tailored to superconducting devices. This field of research has attracted significant attention over the last couple of years, producing a number of approaches tailored to different circuits and architectures. The following sections categorize available approaches by targeted backend architecture and highlight the most notable and innovative solutions. We present the initial problem statement, core contributions, approach, and limitations. Whenever possible, our research is compared with their approaches to highlight our results and further distinguish our contributions. An overview of all discussed approaches is presented in Table 7.1.

After presenting proposed and state-of-the-art approaches, a brief comparison with our implementation is provided in Section 7.5.

7.1 General Purpose Mapping and Routing

The following category of mapping and routing algorithms imposes no constraints on the underlying hardware or the type of circuit. This category is the most studied out of all presented here, but faces severe challenges in the context of near-future error-corrected circuits. Historically, this did not pose any challenges, as QEC received less attention in general.

The goal of most approaches is to minimize the number of additional SWAP gates and the depth of the resulting circuit, while imposing no constraints on the underlying hardware.

7.1.1 SABRE

Zhang et al. proposed SABRE (**S**W**A**P-based **B**idi**RE**ctional heuristic search algorithm), an algorithm widely regarded as the leading approach producing SOTA results both in different gate-related metrics as well as scalability [62]. Given the fast runtime and good results (both experimentally and theoretically), it is generally used as the baseline for mapping and routing algorithms.

Problem Statement. SABRE considers the most general type of quantum circuit; thus, it imposes no constraints on the circuit or hardware topology. The goal is to minimize

Table 7.1: Overview of mapping and routing algorithms for superconducting devices

Type	Name	Comment	QEC	Chiplet	Hardware
Generic	SABRE [62], LightSABRE [15]	Performs poorly for FT circuits and noisy qubits.	×	×	✓
	ML-SABRE [63]	Performs poorly for FT circuits	×	×	✓
	SWin [64]	Performs poorly for FT circuits	×	×	✓
Chiplet	MECH [11]	Does not perform well with surface code.	×	✓	×
	QUBO [65]	Increased runtime for circuits with >50 qubits	×	✓	✓
	Route-Forcing [12]	Does not consider FT circuits and noisy hardware	×	✓	×
	InterPlace [66]	Algorithm not directly applicable to fixed topologies	×	✓	×
	SEQC [67]	Does not consider FT circuits	×	✓	✓
	CCMap [56]	Does not consider FT circuits	×	✓	✓
DQC	HQA [68]	Timeslice valid solutions only	×	×	✓
	pytket-dqc [69]	Does not consider FT circuits	×	×	✓
	DISQCO [50]	Does not consider FT circuits	×	×	✓
QEC	QECC-Synth [14]	Does scale beyond hundreds of qubits	✓	×	✓
	LS Compiler [70]	Does not map to hardware	✓	×	×
	QuIRC [71]	Does not map to hardware	✓	×	×
	EDPC [72]	Does not map to hardware	✓	×	×
	LSBSC [73]	Implementation can generate error corrected circuits for heavy-hex topologies	✓	×	×
	tQEC [13]	Generates circuits for surface code	✓	×	×
	This work		✓	✓	✓

additional SWAP gates and circuit depth.

Contributions and approach. The SABRE algorithm solves the mapping and routing problems using a search algorithm with a heuristic cost function, offering results comparable to those of previous exhaustive mapping-based algorithms while achieving an exponential speedup in search complexity. This is achieved by iteratively optimizing the initial circuit: (1) A random initial mapping is chosen, (2) in order to run the first mapping round. (3) In order to achieve global attention of the circuit, a reverse traversal of the circuit is considered by reversing the order of gates. (4) This step is again followed by a mapping round. Steps (2) - (4) can then be run for many more iterations. The mapping step is performed using a look-ahead SWAP-based search algorithm that employs a heuristic cost function to select the best pair of qubits.

Limitations. While SABRE does not impose any limitations on the underlying hardware topology, it does not work for disjoint QPU setups, as the algorithm assumes that all

physical qubits interact with each other, either directly or indirectly.

7.1.2 LightSABRE

Zou et al. proposed LightSABRE [15], an enhanced version of the SABRE algorithm written in the Rust programming language [15].
Problem Statement. LightSABRE considers the most general type of quantum circuit; thus, it imposes no constraints on the circuit hardware topology. The goal is to minimize the number of additional *SWAP* gates and the circuit depth.
Contributions and approach. One major contribution of LightSABRE is its open-source implementation of the SABRE algorithm in Rust. This shows scalability beyond 10^2 in sub-second execution time with a $200\times$ speedup over the initial version first presented in [62]. It is implemented in the most recent version of Qiskit [16], and often serves as the baseline for when evaluating new approaches.
Limitations. Since LightSABRE does not implement any major solutions to the limitations of the SABRE algorithm, it inherits the same limitations (apart from disjoint QPU setups).

7.1.3 ML-SABRE

Ping et al. proposed ML-SABRE (**M**ulti**l**evel-SABRE), an enhanced version of the SABRE algorithm [63]. The proposed solution produces higher-quality circuits and improves the runtime of the initial SABRE algorithm for special quantum circuits.
Problem Statement. Similar to SABRE, while trying to improve both the solution quality as well as the runtime of the algorithm.
Contributions and approach. As the name suggests, ML-SABRE solves the mapping and routing problem using a hierarchical (multilevel) optimization approach. The hierarchical optimization approach is realized using a graph coarsening approach on a coupling graph extracted from the initial circuit: (1) The graph coarsening is executed several times in order to incrementally refine the contracted graph. (2) At each iteration, the coarsened graph is mapped using the SABRE algorithm. (3) After reaching a fixed level, iterative uncoarsening is performed, which again consists of mapping of SABRE.
Limitations. As the presented approach is inherently similar to the SABRE algorithm used, it poses the same limitations as the initial version. Since the approach is not implemented in Rust (as with LightSABRE), scalability is severely limited by Python's overhead.

7.1.4 SWin(/+)

Fu et al. proposed SWin(+), a heuristic algorithm focusing on parallel mapping for a wide range of circuits [64].
Problem Statement. SWin(+) introduces a mapping and routing algorithm designed for the same qubit mapping and routing constraints as the SABRE heuristic.
Contributions and approach. This work identifies two greedy patterns that make the usual greedy mappers perform less effectively: Immediate Execution and Online SWAP Insertion. Core aspects of the implementation consist of (1) circuit partitioning, (2) parallel sub-circuit smooth mapping, and (3) parallel circuit connecting. For a full description, the reader is referred to the original implementation in [64], as an in-depth explanation is out of the scope of this work.
Limitations. Due to the complexity of the proposed algorithm, the approach is severely limited in terms of scalability. In the reported results, a comparison between SWin(+) and LightSABRE showed runtimes up to 400× slower.

7.2 Mapping and Routing for Chiplet Architectures

The following category of mapping and routing approaches proposes efficient algorithms tailored to QPU architectures comprising interconnected QPUs (hereinafter, chiplets). While the underlying hardware consists of many chiplets, this setup assumes that quantum communication across distinct chiplets is possible; thus, two qubits from distinct chiplets can interact using generic quantum gates.

While the generic mapping and routing algorithms focus on minimizing *intra-chiplet* (on-chip) SWAP gates and circuit depth, chiplet-based algorithms generally also focus on the minimization of costly *inter-chiplet* gates.

7.2.1 MECH

Zhang et al. proposed MECH, a **M**ulti-**E**ntry **C**ommunication **H**ighway mechanism that implements a communication highway across chiplets in order to enable concurrent inter-chiplet operations [11].
Problem Statement. In general, the problem statement of approaches such as SABRE is valid for the chiplet approaches. Due to the high cost of inter-chiplet connections, MECH extends the problem formulation to minimize the number of operations that use them.
Contributions and approach. MECH proposes three major contributions in order to solve the mapping and routing problem for chiplet architectures: (1) Routing of the circuit is split into two parts: local routing consisting of on-chip routing and

highway entry routing; as well as highway routing for finding the optimal highway path for each highway gate (inter-chiplet gate). (2) Inter-chiplet operations utilizing the so-called highways are performed in a measurement-based manner [74], while on-chip computations remain in the purely gate-based manner. (3) As inter-chiplet operations are more costly (in terms of operational noise level) than on-chip operations, a *effective number of (CNOT) gates* metric is introduced. This metric penalizes inter-chiplet operations in order to be more meaningful than a version that does not account for such penalties in a chiplet setup.
Limitations. The paper evaluated the implementation across different circuit classes for chiplet architectures of varying sizes, demonstrating good performance. However, for the class of error-corrected circuits, the method performs poorly as became visible in Section 6.3.

7.2.2 Route-Forcing

Escofet et al. proposed Route-Forcing, an iterative algorithm that focuses on scalable chiplet architectures, by accounting for the fidelity of each added operation to maximize the mapped circuit's success probability [12].
Problem Statement. Similar to MECH, Route-Forcing tries to minimize inter-chiplet operations.
Contributions and approach. To perform mapping and routing, Route-Forcing uses a heuristic search algorithm that leverages metrics tailored to chiplet architectures. This iterative algorithm minimizes the cost (as calculated by a specified metric) between interactive qubits. This metric consists of two major parts: (1) *attraction force* between interacting qubits, and (2) cost of utilized inter-chiplet links for realizing inter-chiplet operations. For an in-depth description of the proposed algorithm, the reader is referred to the description in [12]
Limitations. While Route-Forcing achieves better circuit quality and shorter compilation time than SABRE for small initial circuits, it struggles with scalability. For circuits with around 1000 qubits, utilizing the Route-Forcing algorithm results in execution times of up to 10^4 seconds.

7.2.3 InterPlace

Du et al. proposed InterPlace, an experimental framework for determining where and how to place inter-QPU couplers for manufacturers [66]. While it is not strictly a mapping and routing algorithm, we briefly described their approach to analyzing inter-chiplet link selection and utilization, along with the metrics they presented.

Problem Statement. Unlike the previously introduced approaches, which assumed a fixed backend topology, InterPlace does not. In addition to optimizing a circuit, a synthetic hardware topology is optimized. Thus, this framework supports the analysis of inter-chip connectivity on both real and synthetic topologies.
Contributions and approach. InterPlace proposes three major contributions: (1) The inter-chip coupler selection is formulated as a co-design problem integrating the topology with communication objectives (qubit interaction). (2) Their approach incorporates topological constraints during the link-selection process (selection of inter-chiplet gates). (3) The proposed cost model takes hardware constraints and practical circuit execution into consideration by penalizing overloaded qubits, favoring low error connections, and utilizing an InterPlace cost function consisting of sparsity penalty, qubit overload penalty, congestion penalty, effective path length, and average path length.
Limitations. Exploring candidate inter-chip connections and compiler behaviors can require several hours per system size. Furthermore, this framework is generally not designed to map generic circuits; it can not be used directly to map circuits to chiplet architectures.

7.3 Mapping and Routing for DQC Architectures

The following category of approaches proposes efficient algorithms tailored to DQC architectures consisting of multiple QPUs (also commonly referred to as cores) connected via either classical channels or links capable of sharing purely quantum information (e.g., to distribute pairs of entangled qubits). Since, in general, it is assumed that QPUs could be located at geographically separate sites, remote inter-core communication is more expensive in both time and noise than local communication. While it was still possible to execute quantum operations across chiplets/QPUs in Section 7.2, this is no longer possible in this section.

While DQC architectures are not considered in the backend evaluation of Chapters 5 and 6, available approaches are presented here for completeness.

7.3.1 HQA

Escofet et al. proposed HQA (**H**ungarian **Q**ubit **A**ssignment), an approach for reducing the mapping and routing problem to a polynomial-time assignment problem utilizing the Hungarian algorithm [68, 75].
Problem Statement. The HQA algorithm seeks an optimal mapping and routing for distinct timeslices and thus does not provide a globally valid solution.

Contributions and approach. The paper presents an approach that is not based on partitioning algorithms. The proposed approach is executed as follows: In the first step, the circuit is split into distinct timeslices. During each timeslice, two-qubit operations that are currently assigned to different cores are extracted. For each of these operations, a value is calculated using a metric, which is then used in the Hungarian algorithm to assign each operation to a core with minimal total cost.
Limitations. Since the HQA algorithm distributes two-qubit operations across cores, its output will be a valid assignment for only a particular timeslice.

7.3.2 pytket_dqc

Andres-Martinez et al. proposed pytket_dqc, an extension to the pyTKET framework [20] for performing mapping and routing on generic DQC setups [69]. By utilizing a hypergraph partitioning framework, assignments of qubits to cores with minimal inter-core communication are produced.
Problem Statement. Propose an algorithm for a DQC setup that minimizes expensive inter-core communication. Unlike HQA, the algorithm's result should be valid across the full circuit rather than at distinct timeslices.
Contributions and approach. The main contribution of pytket_dqc is the presentation of an algorithm that uses a hypergraph partitioner. Their approach consists of three main steps: (1) The initial circuit is translated into a hypergraph, with hyperedges representing interaction between qubits. (2) The hypergraph is partitioned into a set number of partitions (cores) such that the number of cut hyperedges (number of inter-core communications) is minimized. A framework such as KaHyPar [52] is utilized. (3) The resulting partition is then translated to a distributed circuit setup, where hyperedges crossing partitions (cores) correspond to inter-core gates that need to be implemented via Bell pair consumption (or similar approaches). For a more in-depth presentation of the approach, the reader is referred to [69].
Limitations. Partitioning approaches generally struggle with limited connectivity, since general hypergraph partition assumes all-to-all connectivity of the partitions (cores)

7.3.3 DISQCO

Burt et al. proposed DISQCO (**Dis**tributed **Q**uantum **C**ircuit **O**ptimisation), an approach based upon a modified Fiduccia–Mattheyses (FM) algorithm for partitioning generic circuits similar to pytket_dqc [50, 69].
Problem Statement. Similar to pytket_dqc, this approach proposes a mapping and routing algorithm for DQC setups that aims to minimize inter-core communication.

Contributions and approach. DISQCO follows a similar approach to pytket_dqc: transpilation, graph conversion, gate grouping, coarsening, partitioning, refinement, and circuit extraction. It is in the step of the great grouping in which both approaches differ. While pytket_dqc utilizes KaHyPar [52] for partitioning, DISQCO utilizes a custom multilevel partitioning algorithm based on the Fiduccia–Mattheyses heuristic algorithm [76]. By utilizing a multilevel approach that combines coarsening and uncoarsening, the approach improves over pytket_dqc in terms of inter-core communication.
Limitations. Since DISQCO is also constructed using a general hypergraph partitioner, all-to-all connectivity of the partitions (cores) is assumed.

7.4 Mapping and Routing for FT Circuits

In the following section, approaches specifically designed for error-corrected circuits are presented. While all algorithms share similar goals, they are generally not directly comparable, as many approaches assume simplifying assumptions about the initial problem statement. In addition, not all approaches generate a circuit that can be executed on hardware.

7.4.1 QECC-Synth

Yin et al. proposed QECC-Synth, an automated compiler that maps error-correction circuits defined using stabilizer codes onto hardware with sparse connectivity [14].
Problem Statement. QECC-Synth formulates the mapping and routing problem as a MaxSAT optimization to minimize circuit depth and two-qubit gate overhead.
Contributions and approach. The main contributions of QECC-Synth are the use of a SAT solver to solve the mapping and routing problem for generic stabilizer codes as input. Their approach can be split into two steps: (1) Mapping using a MaxSAT formulation, and (2) gate scheduling/routing using a SAT formulation.
Limitations. Since QECC-Synth operates on a stabilizer code as input, it cannot directly accept a circuit. Furthermore, the presented approach is severely limited by the utilized SAT solver in terms of scalability, as it does not scale above >1000 qubits or chiplet setups.

7.4.2 Liblsqecc

Watkins et al. proposes Liblsqecc, a lattice surgery based compiler translating arbitrary circuits into error-corrected circuits using the surface code and lattice surgery instructions [70].

Problem Statement. The presented approach does not perform mapping on hardware, but on a simplified grid. Instead, it performs large-scale lattice surgery with a focus on scalability on a simplified grid layout.
Contributions and approach. Major contributions of the proposed approach include a high-performance compiler capable of efficiently handling tens of millions of operations as well as a flexible resource-estimation and layout framework. By utilizing a custom intermediate representation for lattice surgery instructions, practical analysis of large-scale, error-corrected circuits is possible.
Limitations. While the proposed approach excels in scalability, the compiler itself does not generate a circuit. Rather, it produces instructions for lattice surgery. Thus, it is not possible to use the output directly in simulations.

7.4.3 EDPC

Beverland et al. proposes EDPC (**E**dge-**D**isjoint **P**ath **C**ompilation), a lattice surgery based compiler reducing the routing cost over prior lattice surgery compilers [72].
Problem Statement. The proposed approach solves the surface code compilation problem to generate an optimal set of lattice surgery instructions.
Contributions and approach. Implements long-range connections by preparing long-range Bell pairs on connected qubits. For qubits separated by k qubits, this results in a solution of depth 2, as opposed to depth $2\lceil\frac{k-1}{2}\rceil$ for a standard SWAP-based implementation.
Limitations. While the proposed implementation generates optimized surgery instructions, it does not produce a circuit that can be directly mapped to a hardware backend.

7.4.4 tQEC

tQEC is an open-source, community-driven design automation software library for generating, visualizing, and compiling lattice-surgery instructions for surface code [13].
Problem Statement. The proposed library solves the surface code circuit generation problem by generating error-corrected circuits protected by the rotated surface code.
Contributions and approach. The library is one of the only libraries capable of generating efficient circuits protected by the surface code, including the placement of detectors necessary for performing simulations
Limitations. The current implementation focuses on Clifford operations and does not yet incorporate magic state distillation for T-gates or automated lattice surgery protocols necessary for large-scale circuit generation.

7.5 Comparison

The partitioning approach presented in this thesis was based on the implementation of key algorithms commonly used in both DQC and chiplet approaches. Following the methodologies in Route-Forcing [12] and InterPlace [66], our routing procedure uses similar metrics to penalize inter-chiplet operations and accounts for the noise level of inter-chiplet links. While our implementation and utilization diverge significantly, we share the fundamental objective of minimizing inter-chiplet link usage.

8 Summary and Conclusion

This thesis proposes a compiler framework capable of performing scalable mapping and routing of FT quantum circuits onto chiplet-based quantum architectures while accounting for realistic hardware constraints. By decoupling the high-level fault-tolerant quantum circuit compilation - such as with lattice surgery - from backend-specific mapping and routing decisions, our proposed implementation supports rapid re-targeting across various superconducting architecture configurations and avoids unnecessary recompilation with changing backends.

Thorough evaluation shows our implementation achieves significant improvement of hardware utilization, a *speedup* of up to $20\times$ compared to the SOTA LightSABRE algorithm, with an average *decrease* of *86.39% in circuit depth*, and *91.4% in SWAP gate counts* across multiple sizes of lattice surgery circuits. We believe our approach represents a crucial step in bridging the gap between modular chiplet architectures and fault-tolerant quantum computing.

8.1 Code Availability

All implementations, evaluations, experiments, and source code presented in this thesis are provided as a repository at the following location:
`https://github.com/Wegii/qecc_mapping`

9 Future Work

The approach presented in this thesis is provided as a modular implementation, allowing for easy addition and changes. This allows for a multitude of directions for future work and research. In the following sections, an overview of potential topics for improvements and further experiments is presented.

9.1 Circuit Generation

To evaluate the performance and usability of an implementation, it is advantageous to use a wide range of quantum circuits. To date, several circuit benchmarking tools exist, tailored to both general and fault-tolerant quantum circuits [77, 78]. To use our tool for large-scale circuits, the partitioning stage must be provided with predefined partition assignments that map virtual circuit qubits to QEC code patches. Since this important metadata is not specified in the aforementioned tools, their utilization is not straightforward. Thus, for future evaluation and experimentation with additional circuits, we envision extending these tools to provide additional necessary information (such as partition maps).

In addition to using more circuits, the evaluation could benefit greatly from the use of more QEC codes beyond the surface code. Currently, we heavily rely on surface code and lattice surgery. Unfortunately, to this date, no other libraries similar to tqec [13] provide the functionality of circuit generation and detector placement.

While our evaluation is based on the surface code, expanding the circuit constructions to include a broader range of QEC codes is straightforward and would enhance the robustness of the presented findings. At the moment, our methodology relies heavily on surface codes and lattice surgery, since no alternative frameworks to tqec [13] exist for generating circuits and placing detectors.

9.2 Backend Construction

A future direction for architectural topologies could be extending *GenericChipletBackendV2* and the framework to support more complex network topologies beyond the

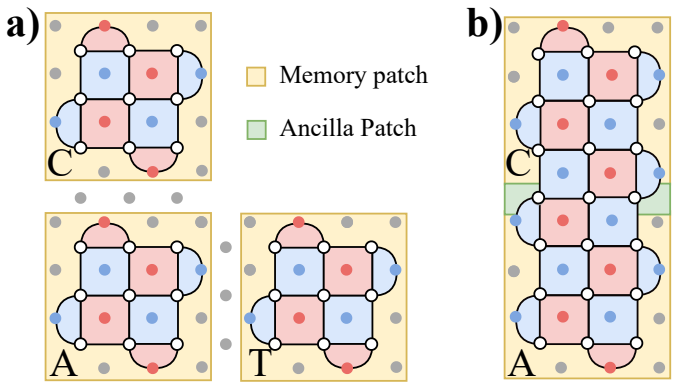

Figure 9.1: Overview of patches in a lattice surgery circuit. ***(a)*** *Three memory patches without any interaction between Control, Ancilla and Target patch.* ***(b)*** *Merge operation on patch C and A, requiring an additional ancilla patch consisting of three qubits.*

current grid and line configurations. Such extensions may require adapting the implemented *partition-to-chiplet* algorithm implemented in the mapping stage.

An additional direction for future architectural configurations is the transition from multi-chiplet systems to DQC setups, as well as the combination of multi-chiplet and DQC setups. This would provide valuable insights into the interaction of FT circuits and modular setups.

9.3 Algorithmic Improvements

9.3.1 Partitioning

We currently face challenges in accurately partitioning lattice surgery circuits. While our system reliably detects memory patches in circuits composed exclusively of them, it inconsistently identifies the ancilla patches required for lattice surgery operations visualized in Figure 9.1. To address this, we have implemented an option to utilize pre-defined partitions.

Although we hope that this will not be necessary in the future, manual partitioning remains necessary in the near term. This necessity arises because current high-level compilers do not yet pass partition data down to the low-level compilers responsible for mapping and routing.

To improve automated detection, one potential solution is to analyze qubit interactions. By examining the connectivity and timing of these interactions, we expect to

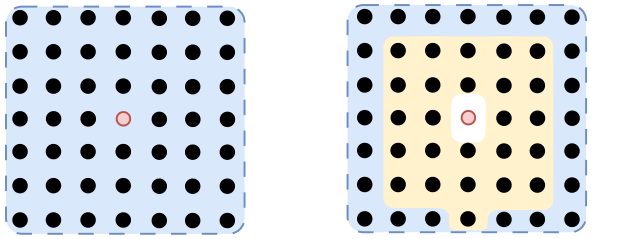

Figure 9.2: Visualization of the improved placement strategy. ***(a)*** *An empty chiplet consisting of a single defective qubit, rendering the simple placement of patches of size 5 impossible.* ***(b)*** *Improved placement strategy performing boundary deformation to incorporate the defective qubit into the patch, enabling the placement of patches of size 5.*

make the underlying ancilla patches visible and distinguishable from static memory patches.

9.3.2 Defective Connections

Currently, our backend definition only considers cases where defective qubits and defective connections are coupled. However, depending on the specific hardware architecture, defects can occur independently. To address this, we propose three primary areas of development for future work. **(1)** The backend should be extended to distinguish between defective qubits and independent defective connections, as suggested by Debroy et al. [79]. **(2)** The mapping algorithm potentially needs modification to implement additional "no-placement zones" based on the location of these defective connections. This ensures that the global connectivity is maintained. Further research is required to verify this claim. **(3)** The routing algorithm must be extended to account for these edge failures. This should be a relatively straightforward adjustment: the defective connections simply need to be removed from the defective coupling map so they are treated as non-existent edges during pathfinding.

9.3.3 Space Efficient Mapping

Rather than placing patches around defective qubits, an alternative approach is to incorporate the defects directly into the patch geometry by constructing the patch geometry around them [80]. This improved strategy is illustrated in Figure 9.2.

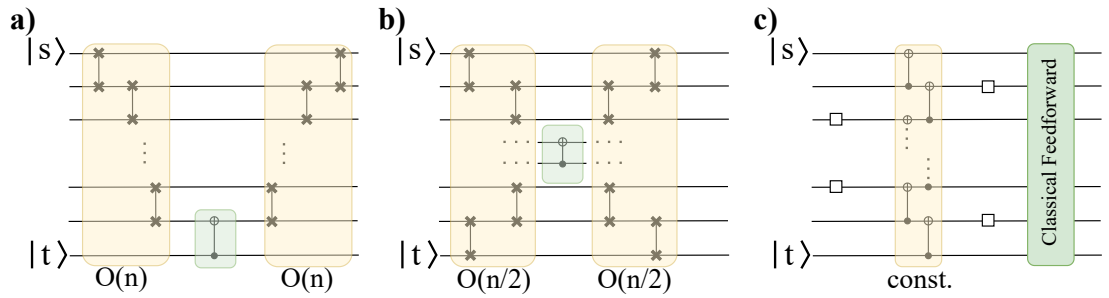

Figure 9.3: Comparison of routing approach for implementing a remote CNOT gate. The choice of approach impacts circuit depth and gate overhead. ***(a)*** *Brute-force SWAP-based approach.* ***(b)*** *Currently implemented approach that reduces depth overhead by 50%.* ***(c)*** *Constant-depth approach based on classical feedforward requiring dynamic circuits.*

While we anticipate that boundary deformation in memory patches will increase the logical error rate, this may be offset by improved chiplet utilization. Higher chiplet utilization can significantly reduce the length of routing paths, thereby lowering the total error rate of the operation. Consequently, further simulation is required to evaluate the trade-off between the physical error overhead of deformed patches and the benefit of reduced routing overhead.

9.3.4 Constant Depth Routing

Realizing long-range gates typically follows one of two primary strategies: heuristic SWAP insertion or measurement-based approaches. The conventional method involves routing qubits by inserting a sequence of SWAP gates. While this is straightforward, this can lead to significant increases in circuit depth and gate overhead. Alternatively, measurement-based techniques similar to those described by Beverland et al. can be used [72]. This approach uses Bell pairs to achieve depth-2 swapping. The trade-offs between these methods are visualized in Figure 9.3.

We expect the benefit of the constant-depth approach to be distance-dependent: while it excels over long paths, the classical feed-forward and entanglement overhead can incur worse results for shorter routing distances. To assess the practical utility of this approach, it is necessary to analyze the path lengths inherent in FT circuits. Our current hypothesis suggests these paths are predominantly short, potentially favoring optimized SWAP-based methods over complex dynamic circuits.

9.4 Performance Improvements

Future large-scale circuits will consist of 100k-1M qubits and high circuit depths. Thus, a next-generation compiler needs to handle large-scale circuits. While the implementation presented in this thesis can outperform the baseline algorithm, further performance improvements are possible.

During development, a multi-threaded version of the utilized routing algorithm was tested. Due to an incompatibility of the utilized rustworkx [59] library and the Python v3.12 *global interpreter lock (GIL)*, no improvement could be achieved. This could be explored again using Python >v3.14, since this version supports a free-threading interpreter that disables the GIL. This could unlock improvements enabled by multi-threading. Furthermore, this could be circumvented with a native Rust implementation. While a *Rust* implementation of the routing algorithm has been developed as part of the presented implementation, further development and research would be necessary to improve the overall integration and, thus, performance.

During evaluation, we observed that most time is spent during routing. Future work could further benchmark and profile this stage to improve runtime. The implementation of *local routing* would be an ideal starting point, since simple changes could already drastically improve routing: skip routing entirely if the source and target qubits are in the same patch, since our implementation ensures that no routing is necessary within a patch.

List of Figures

List of Tables

Bibliography

[1] R. Babbush, R. King, S. Boixo, W. Huggins, T. Khattar, G. H. Low, J. R. McClean, T. O'Brien, and N. C. Rubin. *The Grand Challenge of Quantum Applications*. 2025. arXiv: 2511.09124 [quant-ph].

[2] D. A. Abanin et al. "Observation of constructive interference at the edge of quantum ergodicity." In: *Nature* 646.8086 (2025), pp. 825–830. DOI: 10.1038/s41586-025-09526-6. arXiv: 2506.10191 [quant-ph].

[3] A. Ciceri, A. Cottrell, J. Freeland, D. Fry, H. Hirai, P. Intallura, H. Kang, C.-K. Lee, A. Mitra, K. Ohno, D. Pemmaraju, M. Proissl, B. Quanz, D. Rajan, N. Shimada, and K. Yograj. *Enhanced fill probability estimates in institutional algorithmic bond trading using statistical learning algorithms with quantum computers*. 2025. arXiv: 2509.17715 [quant-ph].

[4] T. Piskor, M. Schöndorf, M. Bauer, D. Smith, T. Ayral, S. Pogorzalek, A. Auer, and M. Papič. *Simulation and Benchmarking of Real Quantum Hardware*. 2025. arXiv: 2508.04483 [quant-ph].

[5] R. Acharya et al. "Quantum error correction below the surface code threshold." In: *Nature* 638.8052 (2025), pp. 920–926. DOI: 10.1038/s41586-024-08449-y. arXiv: 2408.13687 [quant-ph].

[6] S. Bravyi, O. Dial, J. M. Gambetta, D. Gil, and Z. Nazario. "The Future of Quantum Computing with Superconducting Qubits." In: *J. Appl. Phys.* 132.16 (2022), p. 160902. DOI: 10.1063/5.0082975. arXiv: 2209.06841 [quant-ph].

[7] K. N. Smith, G. S. Ravi, J. M. Baker, and F. T. Chong. "Scaling Superconducting Quantum Computers with Chiplet Architectures." In: *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 2022, pp. 1092–1109. DOI: 10.1109/MICRO56248.2022.00078.

[8] S. B. Rached, Z. Sun, J. Khan, G. Long, S. Rodrigo, C. G. Almudéver, E. Alarcón, and S. Abadal. *Modeling Quantum Links for the Exploration of Distributed Quantum Computing Systems*. 2025. arXiv: 2505.08577 [quant-ph].

[9] D. Horsman, A. G. Fowler, S. Devitt, and R. V. Meter. "Surface code quantum computing by lattice surgery." In: *New Journal of Physics* 14.12 (Dec. 2012), p. 123011. DOI: 10.1088/1367-2630/14/12/123011.

[10] S. Bravyi, A. Cross, J. Gambetta, D. Maslov, P. Rall, and T. Yoder. "High-threshold and low-overhead fault-tolerant quantum memory." In: *Nature* 627 (Mar. 2024), pp. 778–782. DOI: 10.1038/s41586-024-07107-7.

[11] H. Zhang, K. Yin, A. Wu, H. Shapourian, A. Shabani, and Y. Ding. "MECH: Multi-Entry Communication Highway for Superconducting Quantum Chiplets." In: *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ASPLOS '24. La Jolla, CA, USA: Association for Computing Machinery, 2024, pp. 699–714. ISBN: 9798400703850. DOI: 10.1145/3620665.3640377.

[12] P. Escofet, A. Gonzalvo, E. Alarcón, C. G. Almudéver, and S. Abadal. "Route-Forcing: Scalable Quantum Circuit Mapping for Scalable Quantum Computing Architectures." In: *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Vol. 01. 2024, pp. 909–920. DOI: 10.1109/QCE60285.2024.00110.

[13] tQEC contributors. *tQEC: Topological Quantum Error Correction*. https://github.com/tqec/tqec. 2025.

[14] K. Yin, H. Zhang, X. Fang, Y. Shi, T. S. Humble, A. Li, and Y. Ding. "QECC-Synth: A Layout Synthesizer for Quantum Error Correction Codes on Sparse Architectures." In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ASPLOS '25. Rotterdam, Netherlands: Association for Computing Machinery, 2025, pp. 876–890. ISBN: 9798400706981. DOI: 10.1145/3669940.3707236.

[15] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman. *LightSABRE: A Lightweight and Enhanced SABRE Algorithm*. 2024. arXiv: 2409.08368 [quant-ph].

[16] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta. *Quantum computing with Qiskit*. 2024. arXiv: 2405.08810 [quant-ph].

[17] Q. Q. C. D. Team. *qiskit-qec*. Accessed: 2026-01-21. 2025. URL: https://github.com/qiskit-community/qiskit-qec.

[18] C. Gidney. "Stim: a fast stabilizer circuit simulator." In: *Quantum* 5 (July 2021), p. 497. ISSN: 2521-327X. DOI: 10.22331/q-2021-07-06-497.

[19] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. AkashNarayanan, A. Asadi, J. M. Arrazola, U. Azad, S. Banning, C. Blank, T. R. Bromley, B. A. Cordier, J. Ceroni, A. Delgado, O. D. Matteo, A. Dusko, T. Garg, D. Guala, A. Hayes, R. Hill, A. Ijaz, T. Isacsson, D. Ittah, S. Jahangiri, P. Jain, E. Jiang, A. Khandelwal, K. Kottmann, R. A. Lang, C. Lee, T.

Loke, A. Lowe, K. McKiernan, J. J. Meyer, J. A. Montañez-Barrera, R. Moyard, Z. Niu, L. J. O'Riordan, S. Oud, A. Panigrahi, C.-Y. Park, D. Polatajko, N. Quesada, C. Roberts, N. Sá, I. Schoch, B. Shi, S. Shu, S. Sim, A. Singh, I. Strandberg, J. Soni, A. Száva, S. Thabet, R. A. Vargas-Hernández, T. Vincent, N. Vitucci, M. Weber, D. Wierichs, R. Wiersema, M. Willmann, V. Wong, S. Zhang, and N. Killoran. *PennyLane: Automatic differentiation of hybrid quantum-classical computations*. 2022. arXiv: 1811.04968 [quant-ph].

[20] S. Sivarajah, S. Dilkes, A. Cowtan, W. Simmons, A. Edgington, and R. Duncan. "t|ket,ü©: a retargetable compiler for NISQ devices." In: *Quantum Science and Technology* 6.1 (Nov. 2020), p. 014003. DOI: 10.1088/2058-9565/ab8e92.

[21] A. Cowtan, S. Dilkes, R. Duncan, A. Krajenbrink, W. Simmons, and S. Sivarajah. "On the Qubit Routing Problem." en. In: vol. 135. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, 5:1–5:32. DOI: 10.4230/LIPICS.TQC.2019.5.

[22] C. D. Bruzewicz, J. Chiaverini, R. McConnell, and J. M. Sage. "Trapped-ion quantum computing: Progress and challenges." In: *Applied Physics Reviews* 6.2 (May 2019). ISSN: 1931-9401. DOI: 10.1063/1.5088164.

[23] L. Henriet, L. Beguin, A. Signoles, T. Lahaye, A. Browaeys, G.-O. Reymond, and C. Jurczak. "Quantum computing with neutral atoms." In: *Quantum* 4 (Sept. 2020), p. 327. ISSN: 2521-327X. DOI: 10.22331/q-2020-09-21-327.

[24] K. A. D. D. S. B. H. G. G. A. B. B. B. B. Burridge et al. "A manufacturable platform for photonic quantum computing." In: *Nature* 641 (2024), pp. 876–883.

[25] T. E. Roth, R. Ma, and W. C. Chew. "The Transmon Qubit for Electromagnetics Engineers: An introduction." In: *IEEE Antennas and Propagation Magazine* 65.2 (Apr. 2023), pp. 8–20. ISSN: 1558-4143. DOI: 10.1109/map.2022.3176593.

[26] R. Katsumi, K. Takada, F. Jelezko, and T. Yatsui. "Recent progress in hybrid diamond photonics for quantum information processing and sensing." In: *Communications Engineering* 4.1 (May 2025), p. 85. ISSN: 2731-3395. DOI: 10.1038/s44172-025-00398-2.

[27] M. Field, A. Q. Chen, B. Scharmann, E. A. Sete, F. Oruc, K. Vu, V. Kosenko, J. Y. Mutus, S. Poletto, and A. Bestwick. "Modular superconducting-qubit architecture with a multichip tunable coupler." In: *Phys. Rev. Appl.* 21 (5 May 2024), p. 054063. DOI: 10.1103/PhysRevApplied.21.054063.

[28] M. A. Nielsen and I. L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[29] V. V. Albert and P. Faist, eds. *The Error Correction Zoo*. 2026.

[30] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill. "Topological quantum memory." In: *Journal of Mathematical Physics* 43.9 (Sept. 2002), pp. 4452–4505. ISSN: 0022-2488. DOI: 10.1063/1.1499754. eprint: https://pubs.aip.org/aip/jmp/article-pdf/43/9/4452/19183135/4452_1_online.pdf.

[31] H. Bombin and M. A. Martin-Delgado. "Optimal resources for topological two-dimensional stabilizer codes: Comparative study." In: *Phys. Rev. A* 76 (1 July 2007), p. 012305. DOI: 10.1103/PhysRevA.76.012305.

[32] A. A. Kovalev and L. P. Pryadko. "Quantum Kronecker sum-product low-density parity-check codes with finite rate." In: *Phys. Rev. A* 88 (1 July 2013), p. 012311. DOI: 10.1103/PhysRevA.88.012311.

[33] K. Wang, Z. Lu, C. Zhang, G. Liu, J. Chen, Y. Wang, Y. Wu, S. Xu, X. Zhu, F. Jin, Y. Gao, Z. Tan, Z. Cui, N. Wang, Y. Zou, A. Zhang, T. Li, F. Shen, J. Zhong, Z. Bao, Z. Zhu, Y. Han, Y. He, J. Shen, H. Wang, J.-N. Yang, Z. Song, J. Deng, H. Dong, Z.-Z. Sun, W. Li, Q. Ye, S. Jiang, Y. Ma, P.-X. Shen, P. Zhang, H. Li, Q. Guo, Z. Wang, C. Song, H. Wang, and D.-L. Deng. *Demonstration of low-overhead quantum error correction codes*. 2025. arXiv: 2505.09684 [quant-ph].

[34] J. T. Anderson, G. Duclos-Cianci, and D. Poulin. "Fault-Tolerant Conversion between the Steane and Reed-Muller Quantum Codes." In: *Phys. Rev. Lett.* 113 (8 Aug. 2014), p. 080501. DOI: 10.1103/PhysRevLett.113.080501.

[35] A. G. Fowler and C. Gidney. "Low overhead quantum computation using lattice surgery." In: *arXiv: Quantum Physics* (2018).

[36] M. Beverland, V. Kliuchnikov, and E. Schoute. "Surface Code Compilation via Edge-Disjoint Paths." In: *PRX Quantum* 3 (2 May 2022), p. 020342. DOI: 10.1103/PRXQuantum.3.020342.

[37] D. Litinski. "A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery." In: *Quantum* 3 (Mar. 2019), p. 128. ISSN: 2521-327X. DOI: 10.22331/q-2019-03-05-128.

[38] B. Eastin and E. Knill. "Restrictions on Transversal Encoded Quantum Gate Sets." In: *Phys. Rev. Lett.* 102 (11 Mar. 2009), p. 110502. DOI: 10.1103/PhysRevLett.102.110502.

[39] C. Gidney, N. Shutty, and C. Jones. *Magic state cultivation: growing T states as cheap as CNOT gates*. 2024. arXiv: 2409.17595 [quant-ph].

[40] S. Bravyi and A. Kitaev. "Universal quantum computation with ideal Clifford gates and noisy ancillas." In: *Phys. Rev. A* 71 (2 Feb. 2005), p. 022316. DOI: 10.1103/PhysRevA.71.022316.

[41] J.-S. Kim, A. McCaskey, B. Heim, M. Modani, S. Stanwyck, and T. Costa. "CUDA Quantum: The Platform for Integrated Quantum-Classical Computing." In: *Proceedings of the 60th Annual ACM/IEEE Design Automation Conference*. IEEE Press, 2025, pp. 1–4. ISBN: 9798350323481.

[42] O. Higgott and C. Gidney. "Sparse Blossom: correcting a million errors per core second with minimum-weight matching." In: *Quantum* 9 (Jan. 2025), p. 1600. ISSN: 2521-327X. DOI: 10.22331/q-2025-01-20-1600.

[43] P. Panteleev and G. Kalachev. "Degenerate Quantum LDPC Codes With Good Finite Length Performance." In: *Quantum* 5 (Nov. 2021), p. 585. ISSN: 2521-327X. DOI: 10.22331/q-2021-11-22-585.

[44] A. Cross, A. Javadi-Abhari, T. Alexander, N. De Beaudrap, L. S. Bishop, S. Heidel, C. A. Ryan, P. Sivarajah, J. Smolin, J. M. Gambetta, and B. R. Johnson. "OpenQASM3: A Broader and Deeper Quantum Assembly Language." In: *ACM Transactions on Quantum Computing* 3.3 (Sept. 2022). DOI: 10.1145/3505636.

[45] N. Ezzell, B. Pokharel, L. Tewala, G. Quiroz, and D. A. Lidar. "Dynamical decoupling for superconducting qubits: A performance survey." In: *Phys. Rev. Appl.* 20 (6 Dec. 2023), p. 064027. DOI: 10.1103/PhysRevApplied.20.064027.

[46] U. Foundation. *Jeff*. https://github.com/unitaryfoundation/jeff. 2025.

[47] Y. Mao, Y. Liu, and Y. Yang. "Qubit Allocation for Distributed Quantum Computing." In: *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. 2023, pp. 1–10. DOI: 10.1109/INFOCOM53939.2023.10228915.

[48] D. Herr, F. Nori, and S. J. Devitt. "Optimization of lattice surgery is NP-hard." In: *npj Quantum Information* 3 (2017).

[49] M. Y. Siraichi, V. F. dos Santos, C. Collange, and F. M. Q. Pereira. "Qubit allocation." In: *Proceedings of the 2018 International Symposium on Code Generation and Optimization* (2018).

[50] F. Burt, K.-C. Chen, and K. K. Leung. *A Multilevel Framework for Partitioning Quantum Circuits*. 2025. arXiv: 2503.19082 [quant-ph].

[51] M. E. J. Newman and M. Girvan. "Finding and evaluating community structure in networks." In: *Phys. Rev. E* 69 (2 Feb. 2004), p. 026113. DOI: 10.1103/PhysRevE.69.026113.

[52] S. Schlag. "High-Quality Hypergraph Partitioning." PhD thesis. Karlsruhe Institute of Technology, Germany, 2020.

[53] S. Jakobs. "On genetic algorithms for the packing of polygons." In: *European Journal of Operational Research* 88.1 (1996), pp. 165–181. ISSN: 0377-2217. DOI: https://doi.org/10.1016/0377-2217(94)00166-9.

[54] D. S. Johnson. "Near-optimal bin packing algorithms." In: 1973.

[55] S. Polyakovsky and R. M'Hallah. "An agent-based approach to the two-dimensional guillotine bin packing problem." In: *European Journal of Operational Research* 192.3 (2009), pp. 767–781. ISSN: 0377-2217. DOI: https://doi.org/10.1016/j.ejor.2007.10.020.

[56] Z. Du, S. Kan, S. Stein, Z. Liang, A. Li, and Y. Mao. *Hardware-aware Compilation for Chip-to-Chip Coupler-Connected Modular Quantum Systems*. 2025. arXiv: 2505.09036 [quant-ph].

[57] E. W. Dijkstra. "A note on two problems in connexion with graphs." In: *Numer. Math.* 1.1 (Dec. 1959), pp. 269–271. ISSN: 0029-599X. DOI: 10.1007/BF01386390.

[58] A. Hagberg, P. J. Swart, and D. A. Schult. "Exploring network structure, dynamics, and function using NetworkX." In: Los Alamos National Laboratory (LANL). Jan. 2008.

[59] M. Treinish, I. Carvalho, G. Tsilimigkounakis, and N. Sá. "rustworkx: A High-Performance Graph Library for Python." In: *Journal of Open Source Software* 7.79 (2022), p. 3968. DOI: 10.21105/joss.03968.

[60] A. Świerkowska, J. Pflieger, E. Giortamis, and P. Bhatotia. *ECCentric: An Empirical Analysis of Quantum Error Correction Codes*. 2025. arXiv: 2511.01062 [quant-ph].

[61] C. Gidney, M. Newman, A. Fowler, and M. Broughton. "A Fault-Tolerant Honeycomb Memory." In: *Quantum* 5 (Dec. 2021), p. 605. ISSN: 2521-327X. DOI: 10.22331/q-2021-12-20-605.

[62] G. Li, Y. Ding, and Y. Xie. "Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices." In: *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS '19. Providence, RI, USA: Association for Computing Machinery, 2019, pp. 1001–1014. ISBN: 9781450362405. DOI: 10.1145/3297858.3304023.

[63] S. Ping, N. Sathishkumar, W.-H. Lin, H. Wang, and J. Cong. *A High-Performance Multilevel Framework for Quantum Layout Synthesis*. 2025. arXiv: 2505.24169 [quant-ph].

[64] H. Fu, M. Zhu, F. Chen, C. Zhang, J. Wu, W. Xie, and X.-Y. Li. "Effective and Efficient Parallel Qubit Mapper." In: *Trans. Comp.-Aided Des. Integ. Cir. Sys.* 44.5 (May 2025), pp. 1774–1787. ISSN: 0278-0070. DOI: 10.1109/TCAD.2024.3500784.

[65] M. Bandic, L. Prielinger, J. Nublein, A. Ovide, S. Rodrigo, S. Abadal, H. van Someren, G. Vardoyan, E. Alarcon, C. G. Almudever, and S. Feld. " Mapping Quantum Circuits to Modular Architectures with QUBO." In: *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. Los Alamitos, CA, USA: IEEE Computer Society, Sept. 2023, pp. 790–801. DOI: 10.1109/QCE57702.2023.00094.

[66] Z. Du, P. C. Flores, W. Wei, J. Chen, K. Hua, and Y. Mao. "Optimizing Inter-chip Coupler Link Placement for Modular and Chiplet Quantum Systems." In: 2025.

[67] M. J. Jeng, N. V. Maruszewski, C. Selna, M. Gavrincea, K. N. Smith, and N. Hardavellas. *Modular Compilation for Quantum Chiplet Architectures*. 2025. arXiv: 2501.08478 [quant-ph].

[68] P. Escofet, A. Ovide, C. G. Almudever, E. Alarcon, and S. Abadal. " Hungarian Qubit Assignment for Optimized Mapping of Quantum Circuits on Multi-Core Architectures." In: *IEEE Computer Architecture Letters* 22.02 (July 2023), pp. 161–164. ISSN: 1556-6064. DOI: 10.1109/LCA.2023.3318857.

[69] P. Andres-Martinez, T. Forrer, D. Mills, J.-Y. Wu, L. Henaut, K. Yamamoto, M. Murao, and R. Duncan. "Distributing circuits over heterogeneous, modular quantum computing network architectures." In: *Quantum Science and Technology* 9.4 (Aug. 2024), p. 045021. DOI: 10.1088/2058-9565/ad6734.

[70] G. Watkins, H. M. Nguyen, K. Watkins, S. Pearce, H.-K. Lau, and A. Paler. "A High Performance Compiler for Very Large Scale Surface Code Computations." In: *Quantum* 8 (May 2024), p. 1354. ISSN: 2521-327X. DOI: 10.22331/q-2024-05-22-1354.

[71] C. Guinn, S. Stein, E. Tureci, G. Avis, C. Liu, S. Krastanov, A. A. Houck, and A. Li. *Co-Designed Superconducting Architecture for Lattice Surgery of Surface Codes with Quantum Interface Routing Card*. 2023. arXiv: 2312.01246 [quant-ph].

[72] M. Beverland, V. Kliuchnikov, and E. Schoute. "Surface Code Compilation via Edge-Disjoint Paths." In: *PRX Quantum* 3 (2 May 2022), p. 020342. DOI: 10.1103/PRXQuantum.3.020342.

[73] L. S. Herzog, L. Berent, A. Kubica, and R. Wille. *Lattice Surgery Compilation Beyond the Surface Code*. 2025. arXiv: 2504.10591 [quant-ph].

[74] R. Jozsa. *An introduction to measurement based quantum computation*. 2005. arXiv: quant-ph/0508124 [quant-ph].

[75] H. W. Kuhn. "The Hungarian method for the assignment problem." In: *Naval Research Logistics (NRL)* 52 (1955).

[76] C. Fiduccia and R. Mattheyses. "A Linear-Time Heuristic for Improving Network Partitions." In: *19th Design Automation Conference*. 1982, pp. 175–181. DOI: 10.1109/DAC.1982.1585498.

[77] N. Quetschlich, L. Burgholzer, and R. Wille. "MQT Bench: Benchmarking Software and Design Automation Tools for Quantum Computing." In: *Quantum* 7 (2023). MQT Bench is available at https://mqt-bench.app/, p. 1062. DOI: 10.22331/q-2023-07-20-1062. arXiv: 2204.13719.

[78] A. Harkness, S. Kan, C. Liu, M. Wang, J. M. Martyn, S. Xu, D. Chamaki, E. Decker, Y. Mao, L. F. Zuluaga, T. Terlaky, A. Li, and S. Stein. *FTCircuitBench: A Benchmark Suite for Fault-Tolerant Quantum Compilation and Architecture*. 2026. arXiv: 2601.03185 [quant-ph].

[79] D. M. Debroy, M. McEwen, C. Gidney, N. Shutty, and A. Zalcman. "LUCI in the Surface Code with Dropouts." In: *Quantum* 9 (Dec. 2025), p. 1936. ISSN: 2521-327X. DOI: 10.22331/q-2025-12-11-1936.

[80] S. F. Lin, J. Viszlai, K. N. Smith, G. S. Ravi, C. Yuan, F. T. Chong, and B. J. Brown. "Codesign of quantum error-correcting codes and modular chiplets in the presence of defects." In: *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, Apr. 2024, pp. 216–231. DOI: 10.1145/3620665.3640362.